

AI Agent 설계 심화 for Multi Agent

LGCNS Gen.AI 교육

강사 김형욱
(hukim@artiasolution.com)

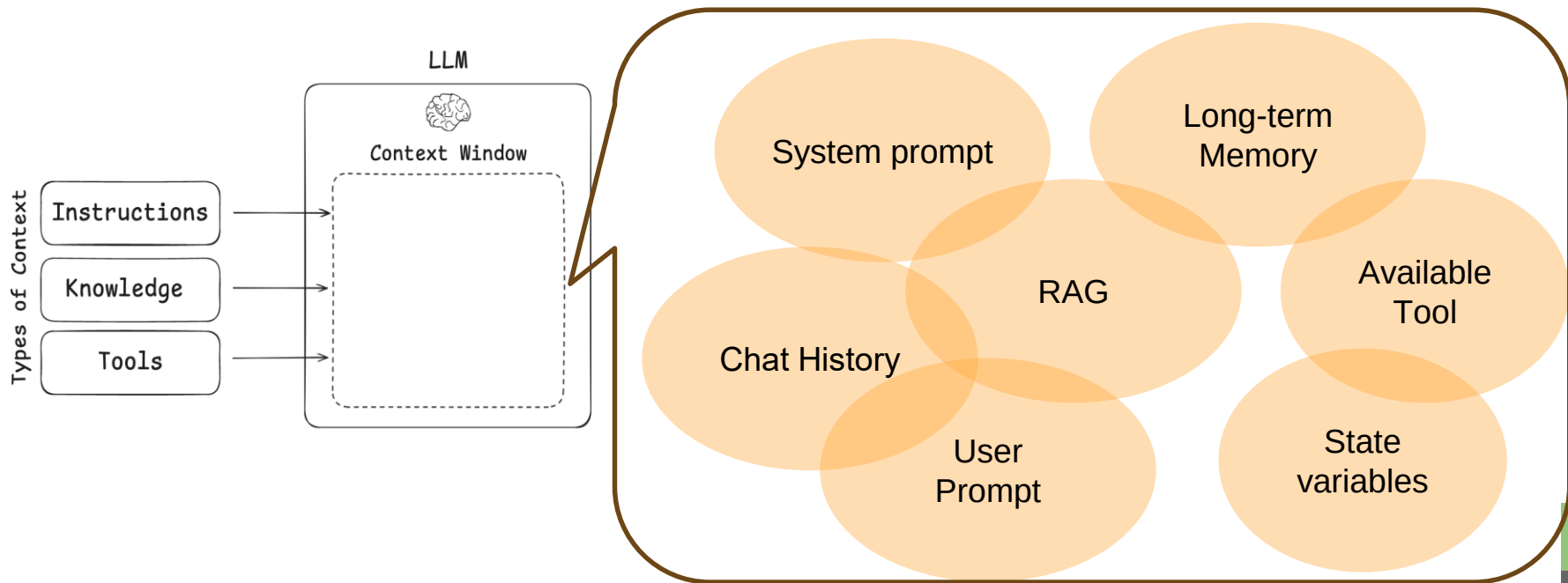
Contents

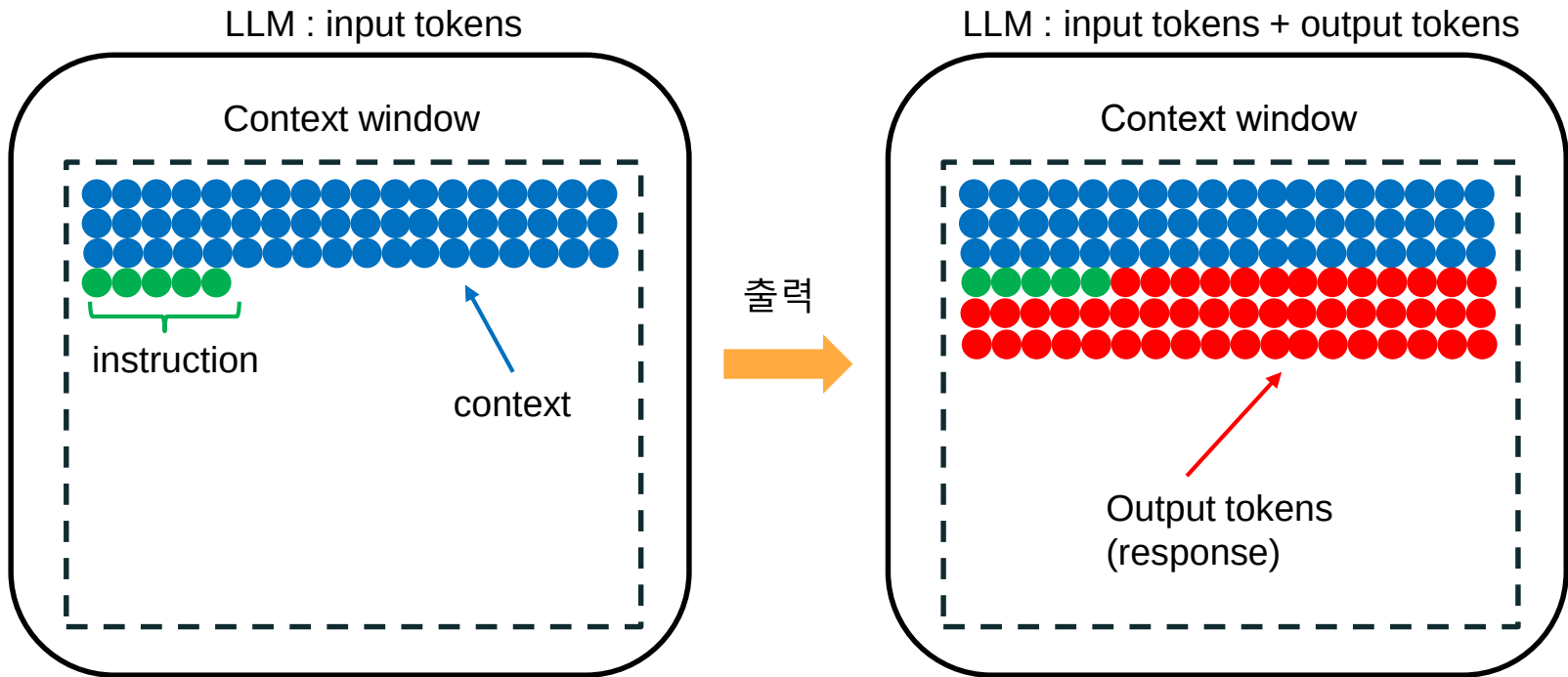
- 멀티 에이전트 개요 (3 page)
- 에이전틱 프레임워크 (21 page)
- 컨텍스트 엔지니어링 (36 page)
- 에이전트 운영 및 평가 (49 page)
- 부록.멀티 에이전트 담론 (70 page)

멀티 에이전트 개요

LLM과 컨텍스트

- 컨텍스트(Context)란 LLM이 참고해야 하는 정보 전체
 - 프롬프트(Prompt)가 현재 턴에 주어진 입력 문장이라면, 컨텍스트는 참고된 모든 배경 정보

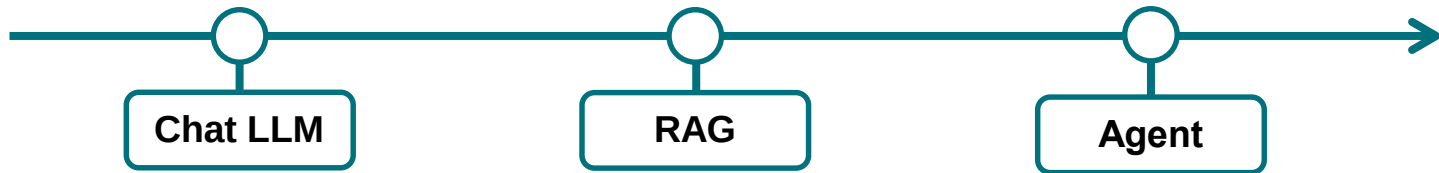




- LLM의 최적화된 작업을 위한 맥락을 Context Window 안에 넣는 것이 목표
- 결국 핵심은 Context

LLM에서 에이전트로 진화

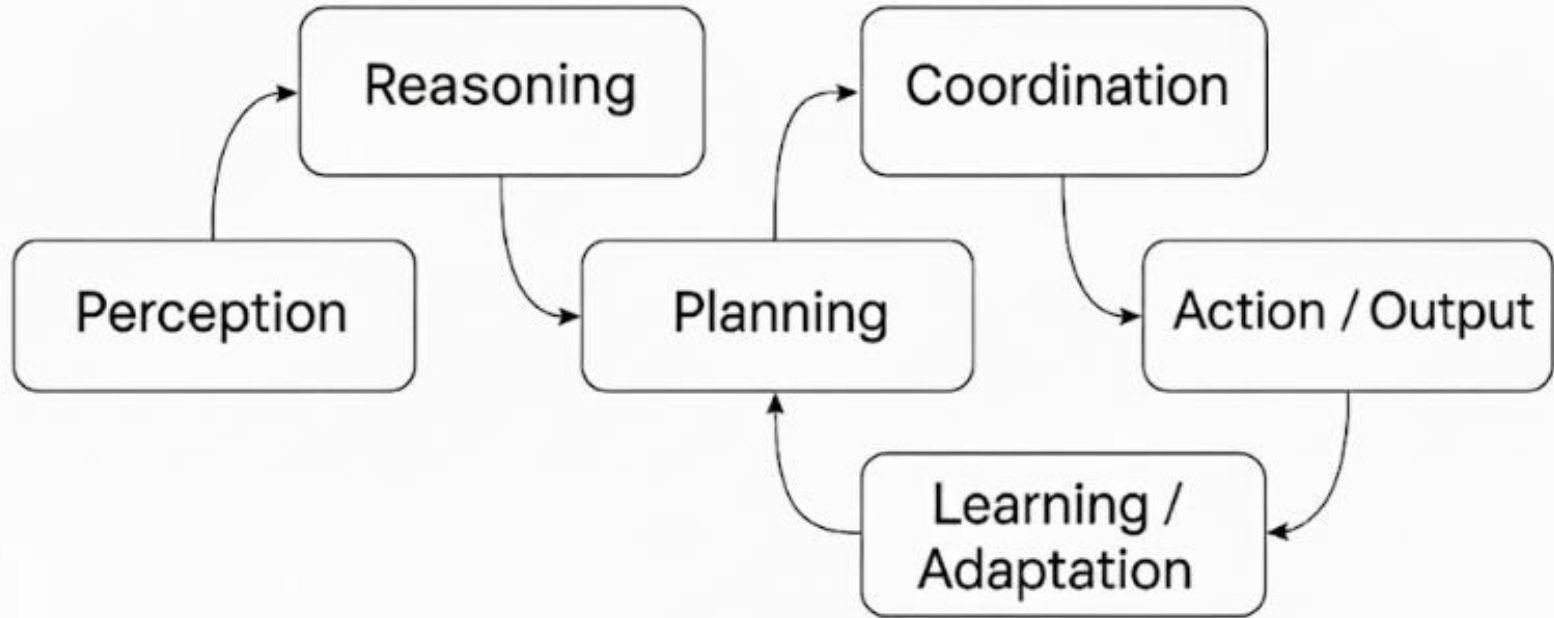
- LLM Application의 진화
 - 1단계: Chat LLM :
 - ✓ 대화 컨텍스트을 이해하는 지능형 챗봇
 - 2단계: RAG (Retrieval Augmented Generation)
 - ✓ 검색 컨텍스트을 활용하는 질의응답 시스템
 - 3단계: 에이전트 (Agent)
 - ✓ 외부 시스템 상호작용하며, 도구 컨텍스트를 활용하며 점진적 문제 해결

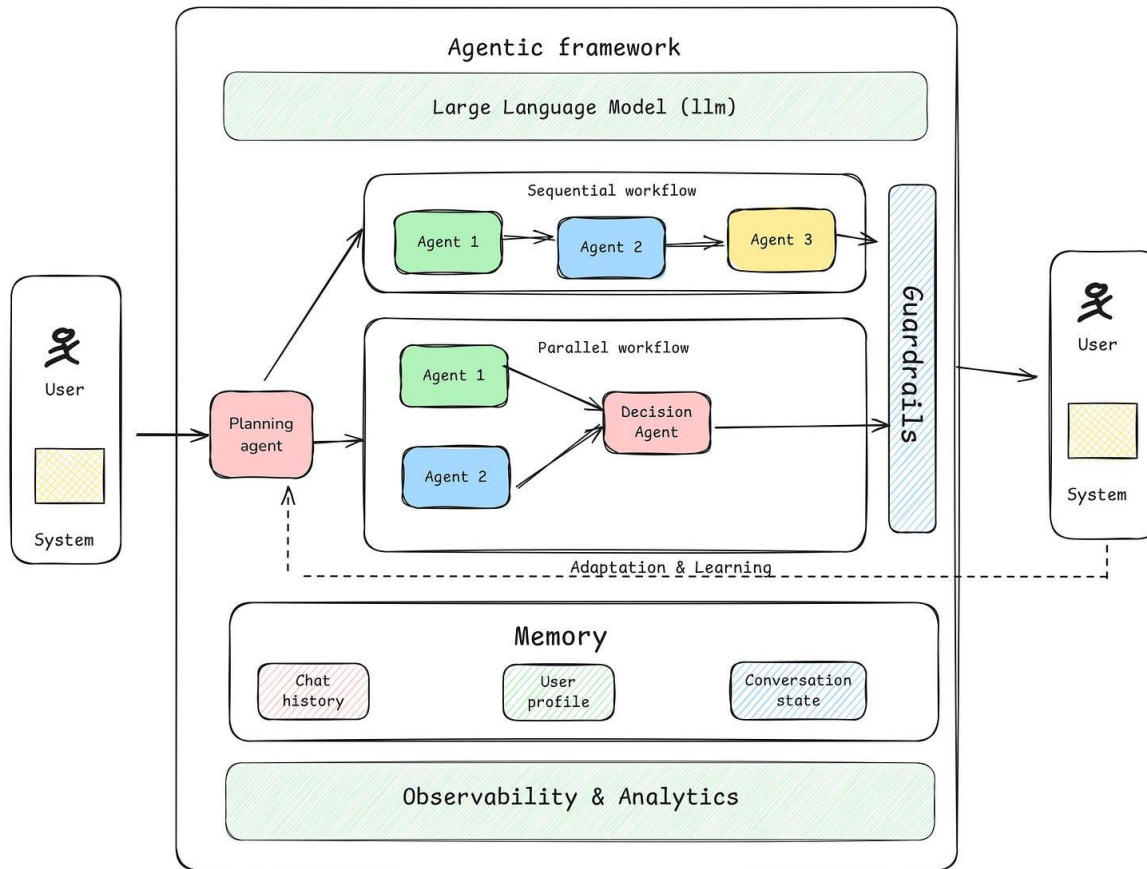


에이전트 AI

- AI 에이전트 더 높은 수준의 자율성 / 문제 해결 능력 / 적응력 요구
 - AI 스스로 전체 워크플로우를 완전 자동화할 수 있는 시스템
- 목표 수행을 위해 스스로 계획·학습하며, 동시에 윤리적이고 고급 맥락 판단 능력 필요
 - 에이전트 관련 고급 기법 동원
 - ✓ 고급 추론 모델
 - ✓ 멀티 에이전트 오케스트레이션
 - ✓ 특정 업무에 전문적인 AI 에이전트
 - ✓ 컨텍스트 엔지니어링
 - ✓ 가드레일 & 모니터링 기술

Agentic AI





Agentic AI 아키텍처

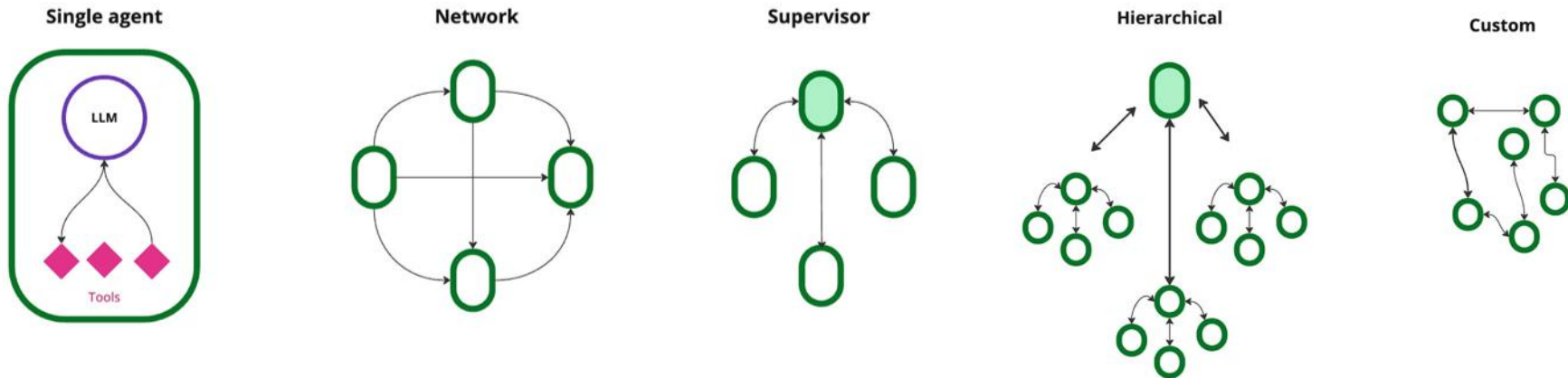
단일 에이전트 한계

- 역할의 과부하
 - 모든 기능을 떠맡아 도구 선택·의사결정 오류 발생
 - 많은 작업량으로 맥락이 방대해지면 작업 경로 추적·정확성 저하
 - 특정 분야에 대한 전문성 부족
- 컨텍스트 실패
 - 컨텍스트 윈도우의 크기 제약으로 정보 처리 부족
 - 컨텍스트 간 충돌, 오염 등
- 멀티태스킹 능력 부족
 - 직렬 처리로 병렬 대응·속도 저하

멀티 에이전트 필요성

- 단일 에이전트의 한계를 극복하기 위해 도입
 - 어려운 복잡한 문제를 다수의 에이전트가 협력하여 해결
 - 개별 에이전트는 독립적으로 동작하면서 필요한 지식과 정보를 공유
- 멀티 에이전트 시스템의 이점
 - 전문화 : 특정 도메인에 집중하는 전문가 에이전트 생성 가능
 - 모듈성 : 기능별로 에이전트가 담당하여 개발, 테스트, 유지 관리가 용이
 - 협업 : 에이전트의 협업을 설계하고, 방식을 효율적으로 제어할 수 있음
 - 컨텍스트 관리 : 컨텍스트 윈도우 포화를 막고 특정 문제에 특화된 지식을 제공 가능

멀티 에이전트 아키텍처

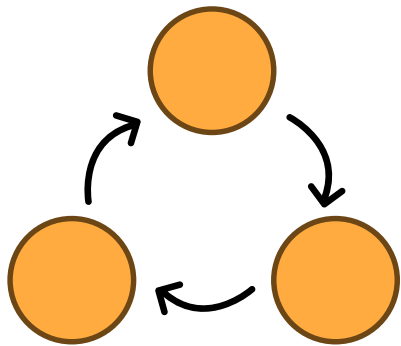


- **단독 에이전트:** 도구 사용 권한을 가진 에이전트가 애플리케이션의 제어 흐름을 결정하는 시스템
- **네트워크:** 각 에이전트가 자유롭게 연결되어 있어 다음 실행 에이전트를 동적으로 선택할 수 있는 유연한 구조
- **지도자:** 중앙의 감독 에이전트가 하위 에이전트를 호출하고 제어하는 방식으로 명확한 흐름과 제어가 가능
- **계층형:** 여러 감독자와 하위 에이전트를 팀 단위로 구성해 복잡한 작업을 다단계로 분담하고 확장성을 높임
- **커스텀:** 에이전트 간 연결 흐름을 명시적으로 정의하거나 일부 흐름만 LLM의 판단에 맡기는 혼합형 구조

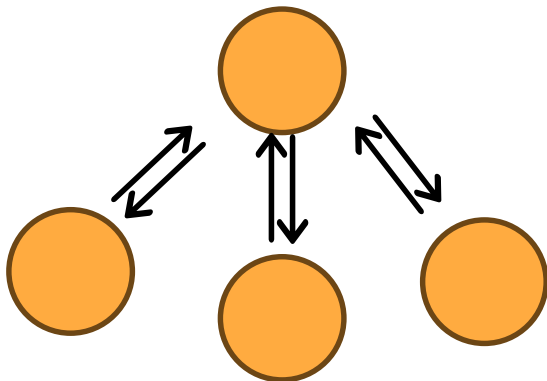
멀티 에이전트의 통신 방식

- 멀티 에이전트는 여러 AI 에이전트가 협력하여 문제 해결
 - 효과적인 협력을 위해 에이전트 간 통신 방식이 핵심
- 에이전트 간 통신의 3가지 핵심 설계 요소
 - 구조 (Topology): 누가 흐름을 제어하는가?
 - 입력(Input): 하위 작업자에게 맥락을 어떻게 전달하는가?
 - 출력(Output): 작업자의 결과를 어디까지 공유하는가?

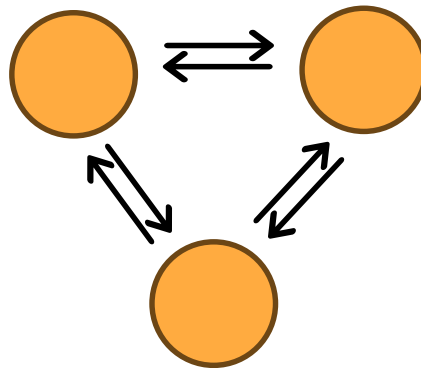
1 구조 : 누가 흐름을 제어하는가?



라운드로빈(Round Robin)



슈퍼바이저(Supervisor)



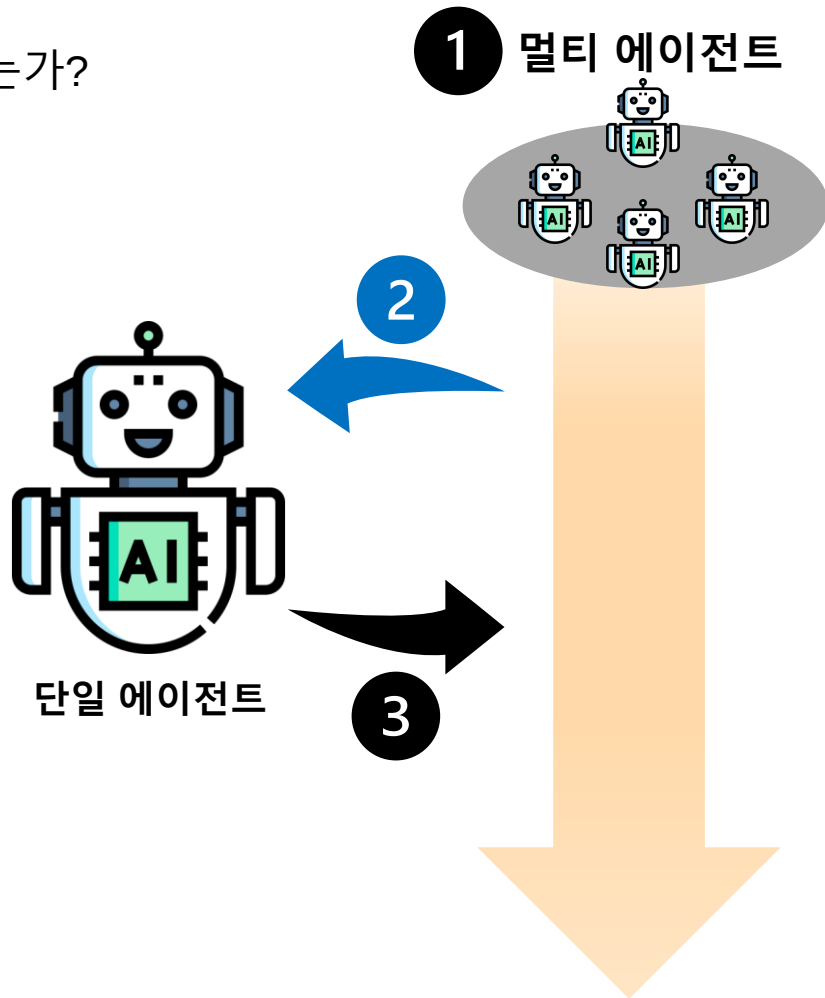
스웜(Swarm)

- 어떤 에이전트(또는 팀)이 작업을 할 것인가를 다음과 같은 방식으로 조정하여 협업
 - Round Robin** : 에이전트들이 고정된 순서로 차례대로 발화하며 작업을 진행
 - Supervisor** : 현재 대화 흐름에 맞춰 가장 적합한 에이전트를 선택하여 다음 작업 진행
 - Swarm** : 모든 에이전트가 서로에게 작업을 위임(handoff)

2 위임 방식 : 입력(Input)을 어떻게 전달하는가?

• Handoff (핸드오프)

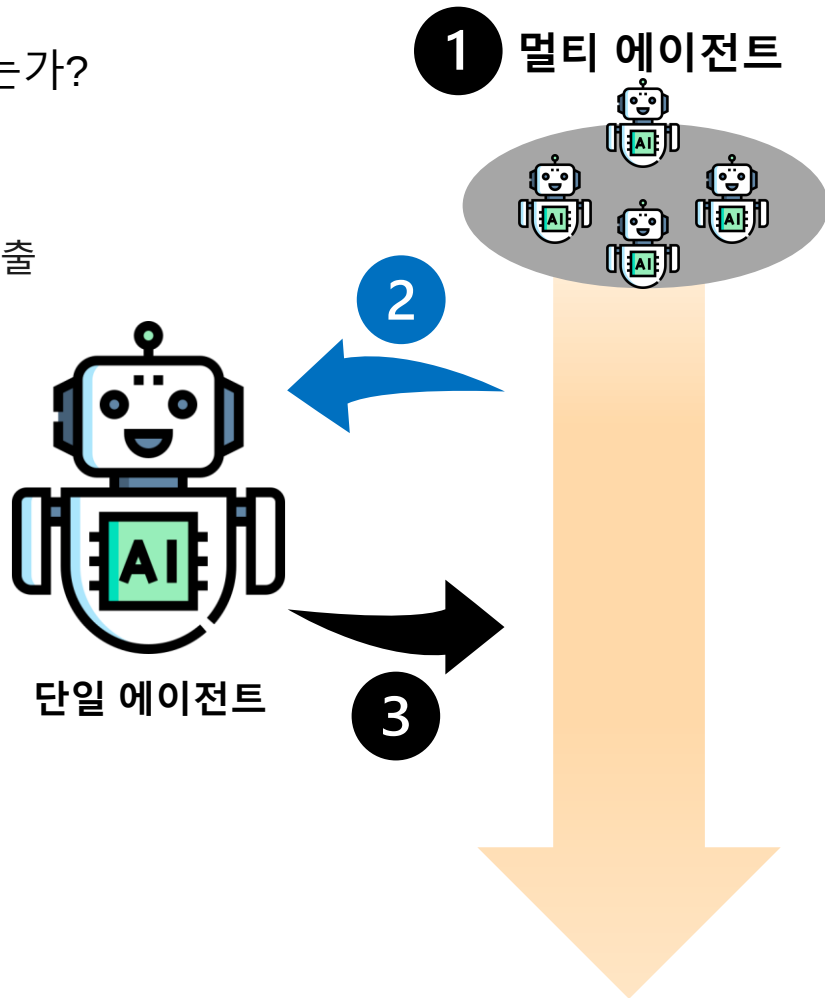
- 다음 에이전트에게 작업 위임
- 특징
 - ✓ 탈중앙화된 네트워크 방식 가능
 - ✓ 전체 대화 기록 공유 + 작업 제어권 이전
- 장점: 👍
 - ✓ 모든 작업자가 전체 맥락을 이해
 - ✓ 유연한 대화 흐름
 - ✓ 인간의 대화 참여 쉬움
- 단점: 👎
 - ✓ 정보 오염 가능
 - ✓ 컨텍스트 포화 빠름
 - ✓ 토큰 비용 비효율



2 위임 방식 : 입력(Input)을 어떻게 전달하는가?

- Agents-as-Tools (에이전트 호출)

- 지휘자 에이전트가 하위 에이전트를 함수로 호출
- 특징
 - ✓ 중앙 집중형 구조 사용
 - ✓ 필요한 정보만 전달
- 장점: 👍
 - ✓ 많은 에이전트를 쉽게 통합
 - ✓ 토큰 효율성 극대화
- 단점: 👎
 - ✓ 중앙집중 강제
 - ✓ 전체 작업에서 지휘자 에이전트의 판단 능력에 의존



When Agent 1 calls Agent 2, what does Agent 2 see?

```
{  
  "messages": ...,  
  "artifacts": ...  
}
```

Agent 1 directly passes
overall state to Agent 2

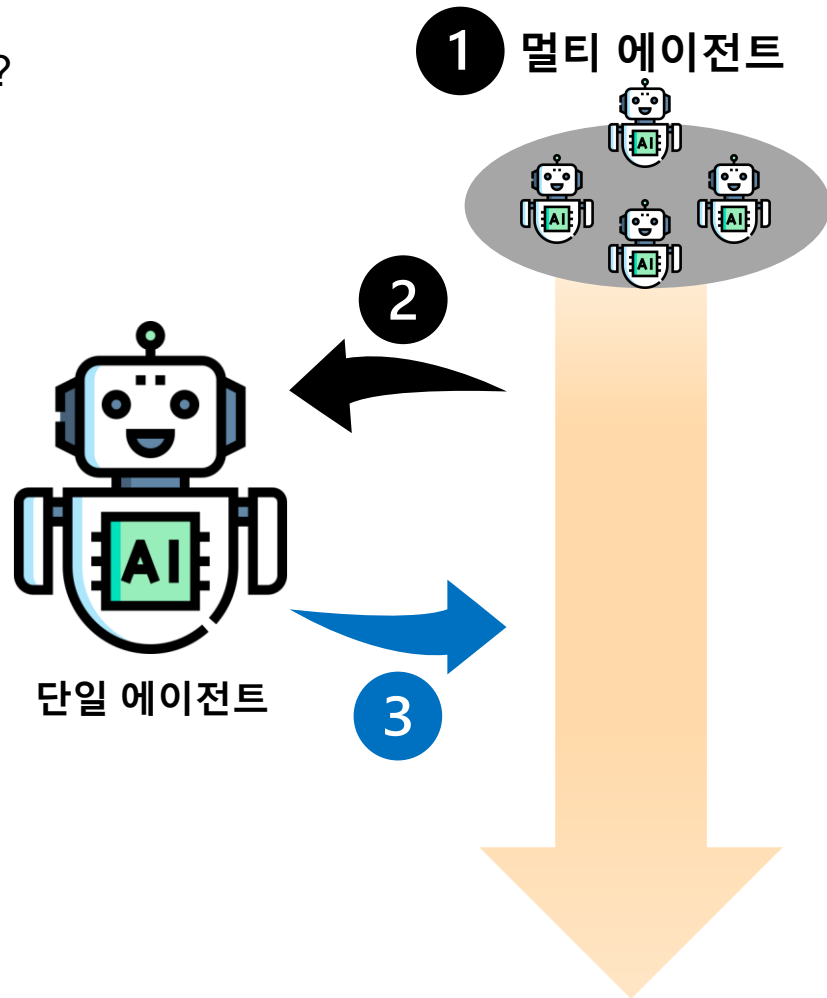
```
{  
  "tool_arg1": ...,  
  "tool_arg2": ...  
}
```

Agent 1 uses an LLM + tools to
determine what Agent 2 sees

3 출력 방식 : 출력을 어디까지 공유하는가?

- 전체 컨텍스트 공유 (Full Context Sharing)

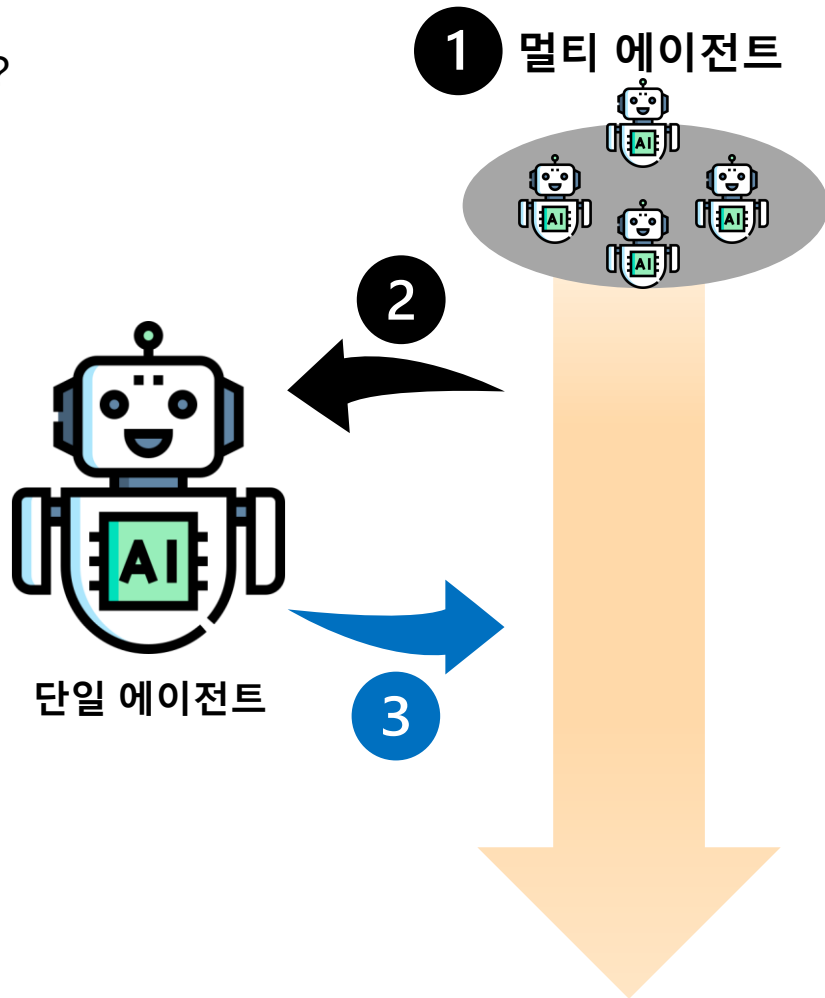
- 모든 에이전트가 대화 전체 히스토리에 접근. 에이전트의 사고 과정이 포함됨.
- 장점: 👍
 - ✓ 풍부한 맥락을 반영 가능
 - ✓ 협업 효율 증가
- 단점: ☹️
 - ✓ 비용 증가, 정보 과잉 위험
 - ✓ 컨텍스트 포화가 빠름



3 출력 방식 : 출력을 어디까지 공유하는가?

- 결과만 공유 (Final Result Only)

- 필요한 결과/요약만 다음 에이전트에 전달
- 특징
 - ✓ 중앙 에이전트가 필요한 컨텍스트만 전달
- 장점: 🧐
 - ✓ 효율적, 불필요한 정보 노출 방지
 - ✓ 가벼운 메시지, 높은 확장성
- 단점: 🧐
 - ✓ 세부 맥락이 손실될 수 있음
 - ✓ 전체 맥락 부족으로 인한 오판 가능성



When Agent 1 calls Agent 2, what does Agent 1 get back in response?

Agent 2 Tool Call

Tool Call

Agent 2 Final Response

Agent 2's final response and
their inner thought process

Agent 2 Tool Call

Tool Call

Agent 2 Final Response

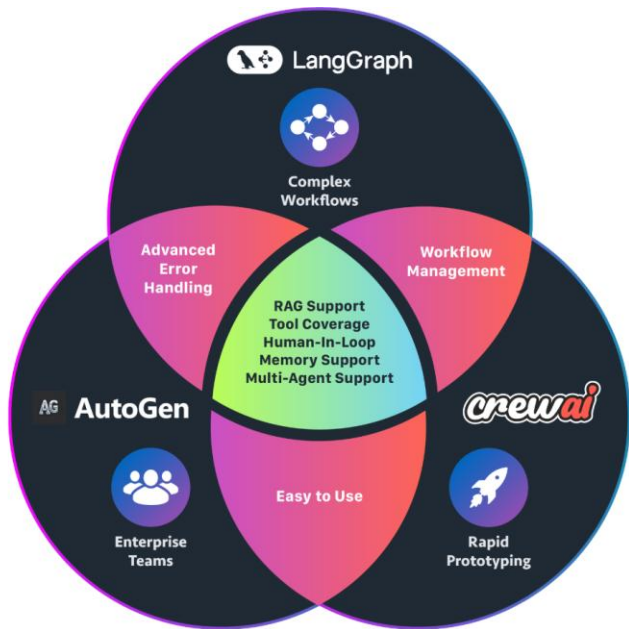
Just Agent 2's final response

Agentic Frameworks

주요 Agentic Frameworks

프레임워크	개발 주체	주요 특징 및 설명
Google ADK	Google	Vertex AI 및 Gemini와 긴밀히 통합되며, 멀티에이전트 및 코드 기반 제어에 강점을 가짐
LangChain	-	대표적인 LLM 기반 프레임워크로, 다양한 도구 연동 및 커뮤니티 지원이 우수함
LangGraph	LangChain 개발팀	작업 흐름을 그래프로 관리하며, 복잡하고 순환적인 에이전트 구조 설계에 적합
AutoGen	Microsoft	대화형 멀티에이전트에 특화되어 있으며, 비동기 기반 협업 처리에 강점
CrewAI	-	역할 기반 에이전트 팀 구성에 초점을 맞추며, 협업 기반 목표 달성에 효과적
Semantic Kernel	Microsoft	.NET 기반으로 개발되어 기존 코드와 자연스러운 통합, 보안·컴플라이언스에 적합
Pydantic AI	Pydantic	Pydantic 모델을 기반으로 안전하고 구조화된 입력/출력을 지원하는 에이전트 구성 도구
Agno	Agno 팀 (오픈소스)	미니멀한 설계와 자연어 기반 구성 방식이 특징이며, 직관적인 프롬프트 기반 워크플로우 구축 가능

LangGraph vs AutoGen vs CrewAI



프레임워크	주요 특징	장단점
LangGraph	그래프 기반 워크플로우 관리	• 복잡한 다단계 프로세스 제어 가능 • 초기 학습 난이도 있음
AutoGen	엔터프라이즈급 AI 에이전트 프레임워크	• 높은 에이전트 추상성 • 에이전트 동작의 정교한 제어 한계
CrewAI	빠른 프로토타이핑 및 쉬운 설정	• 빠른 개발 및 테스트 가능 • 복잡한 워크플로우에는 한계

LangGraph ↔ AutoGen: 호환 가능
(LangGraph로 흐름을 관리하고, AutoGen을 실행 엔진으로 활용 가능)

	LangGraph	AutoGen	CrewAI
통합성	★★★★★	★★★	★★★★
규모화	★★★★☆	★★★	★★★☆
유연성	★★★★★	★★	★★★☆
학습속도	★★	★★★★☆	★★★☆
참고문서 품질	★★★★	★★★☆	★★★★☆
다양한 기능	★★★★	★★☆	★★★☆
배포	★★★★	★★★	★
인터페이스	★★	★★★	★

AutoGen

```
# OpenAI 모델 클라이언트를 생성합니다.  
model_client = OpenAIChatCompletionClient(  
    model="gpt-4o",          # 사용할 모델: gpt-4o  
    temperature=1,          # 생성 다양성 조절 값  
)
```

```
# 주요(primary) 에이전트를 생성합니다.
```

```
primary_agent = AssistantAgent(  
    "primary", # 에이전트 이름  
    model_client=model_client,  
    system_message="당신은 유용한 AI 비서입니다.", # 시스템 역할 메시지  
)
```

```
# 비평(critic) 에이전트를 생성합니다.
```

```
critic_agent = AssistantAgent(  
    "critic", # 에이전트 이름  
    model_client=model_client,  
    system_message="모든 메시지에 대해 건설적인 피드백을 제공하세요.",  
)
```



첫 번째 에이전트

→ 작업자



두 번째 에이전트

→ 비평가

```
max_msg_termination = MaxMessageTermination(max_messages=5)
round_robin_team = RoundRobinGroupChat([primary_agent, critic_agent], termination_condition=max_msg_termination)
```

GroupChat : AutoGen에서 에이전트 팀의 패턴

대화 종료조건

```
----- user -----
서울에 대한 4문장 시를 지어줘
----- primary -----
하늘 아래 무릎 선 남산의 자태,
골목길엔 오래된 이야기가 숨쉬네.
광화문 앞에서 역사가 흐르고,
서울, 그 안에 수많은 삶이 춤추네.
----- critic -----
이번 시도 서울의 매력을 잘 표현하고 있습니다. 첫 줄에서 남산을 통해 서울의 대표적인 랜드마크를 강조하고 있으며, 두 번째 줄은 서울의 오래된 골목길과 그 안에 담긴 이야기를 잘 그려내고
----- primary -----
칭찬해 주셔서 감사합니다! 서울의 다양한 요소들이 조화롭게 드러나도록 노력했는데, 그 점이 잘 전달된 것 같아 기쁘네요. 서울의 매력을 담은 시를 더 원하시거나 다른 주제로 필요하시면 언제든
----- critic -----
제가 드린 피드백을 좋게 받아주셔서 감사합니다. 서울의 매력을 풍부하게 표현하신 점이 정말 인상적이었습니다. 앞으로도 다양한 주제에 대해 시를 쓰실 때, 지금처럼 세심하게 각 요소들을 조
TaskResult(messages=[TextMessage(source='user', models_usage=None, metadata={}, content='서울에 대한 4문장 시를 지어줘', type='TextMessage'), TextMessage(source='primary',
models_usage=RequestUsage(prompt_tokens=340, completion_tokens=55), metadata={}, content='하늘 아래 무릎 선 남산의 자태, \n골목길엔 오래된 이야기가 숨쉬네. \n광화문 앞에서 역사가
흐르고, \n서울, 그 안에 수많은 삶이 춤추네. ', type='TextMessage'), TextMessage(source='critic', models_usage=RequestUsage(prompt_tokens=545, completion_tokens=134), metadata={},
content='이번 시도 서울의 매력을 잘 표현하고 있습니다. 첫 줄에서 남산을 통해 서울의 대표적인 랜드마크를 강조하고 있으며, 두 번째 줄은 서울의 오래된 골목길과 그 안에 담긴 이야기를 잘
그려내고 있습니다. 세 번째 줄은 광화문을 통해 서울의 역사적 중요성을 부각시켰고, 마지막 줄에서는 서울이라는 도시 안에서 다양한 삶의 모습을 생동감 있게 묘사했습니다. 각 요소들이 조화
를 이루며 서울의 다채로운 면모를 효과적으로 나타내고 있어 매우 인상적입니다. 잘하셨습니다!', type='TextMessage'), TextMessage(source='primary',
models_usage=RequestUsage(prompt_tokens=539, completion_tokens=63), metadata={}, content='칭찬해 주셔서 감사합니다! 서울의 다양한 요소들이 조화롭게 드러나도록 노력했는데, 그 점이 잘
전달된 것 같아 기쁘네요. 서울의 매력을 담은 시를 더 원하시거나 다른 주제로 필요하시면 언제든지 말씀해 주세요.', type='TextMessage'), TextMessage(source='critic',
models_usage=RequestUsage(prompt_tokens=752, completion_tokens=106), metadata={}, content='제가 드린 피드백을 좋게 받아주셔서 감사합니다. 서울의 매력을 풍부하게 표현하신 점이 정말
인상적이었습니다. 앞으로도 다양한 주제에 대해 시를 쓰실 때, 지금처럼 세심하게 각 요소들을 조화롭게 담아내시면 좋을 것 같습니다. 시의 주제가 더 필요하시거나 다른 스타일로 시도해보고
싶으실 때 언제든지 말씀해 주세요. 창의적인 작업을 계속 이어가시길 바랍니다!', type='TextMessage')], stop_reason='Maximum number of messages 5 reached, current message count: 5')
```

- **GroupChat 패턴으로 협업형 에이전트 팀을 쉽게 구현 가능**
 - GroupChat 내 개별 에이전트는 지정된 그룹 대화 규칙에 따라 유저 요청을 처리

CrewAI

```
# src/latest_ai_development/config/agents.yaml
researcher:
  role: >
    {topic} Senior Data Researcher
  goal: >
    Uncover cutting-edge developments in {topic}
  backstory: >
    You're a seasoned researcher with a knack for uncovering the latest
    developments in {topic}. Known for your ability to find the most relevant
    information and present it in a clear and concise manner.

reporting_analyst:
  role: >
    {topic} Reporting Analyst
  goal: >
    Create detailed reports based on {topic} data analysis and research findings
  backstory: >
    You're a meticulous analyst with a keen eye for detail. You're known for
    your ability to turn complex data into clear and concise reports, making
    it easy for others to understand and act on the information you provide.
```

```
# src/latest_ai_development/config/tasks.yaml
research_task:
  description: >
    Conduct a thorough research about {topic}
    Make sure you find any interesting and relevant information given
    the current year is 2025.
  expected_output: >
    A list with 10 bullet points of the most relevant information about {topic}
  agent: researcher

reporting_task:
  description: >
    Review the context you got and expand each topic into a full section for a report.
    Make sure the report is detailed and contains any and all relevant information.
  expected_output: >
    A fully fledge reports with the mains topics, each with a full section of information.
    Formatted as markdown without ''''
  agent: reporting_analyst
  output_file: report.md
```

```
# src/latest_ai_development/crew.py
from crewai import Agent, Crew, Process, Task
from crewai.project import CrewBase, agent, crew, task
from crewai_tools import SerperDevTool
from crewai.agents.agent_builder.base_agent import BaseAgent
from typing import List

@CrewBase
class LatestAiDevelopmentCrew():
    """LatestAiDevelopment crew"""

    agents: List[BaseAgent]
    tasks: List[Task]

    @agent
    def researcher(self) -> Agent:
        return Agent(
            config=self.agents_config['researcher'], # type: ignore[index]
            verbose=True,
            tools=[SerperDevTool()]
        )

    @agent
    def reporting_analyst(self) -> Agent:
        return Agent(
            config=self.agents_config['reporting_analyst'], # type: ignore[index]
            verbose=True
        )
```

```
@task
def research_task(self) -> Task:
    return Task(
        config=self.tasks_config['research_task'], # type: ignore[index]
    )

@task
def reporting_task(self) -> Task:
    return Task(
        config=self.tasks_config['reporting_task'], # type: ignore[index]
        output_file='output/report.md'
    )

@crew
def crew(self) -> Crew:
    """Creates the LatestAiDevelopment crew"""
    return Crew(
        agents=self.agents, # Automatically created by the @agent decorator
        tasks=self.tasks, # Automatically created by the @task decorator
        process=Process.sequential,
        verbose=True,
    )
```

LangGraph

```
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from langgraph_supervisor import create_supervisor

def book_hotel(hotel_name: str):
    """호텔 예약"""
    return f"{hotel_name} 숙박 예약이 완료되었습니다."

def book_flight(from_airport: str, to_airport: str):
    """항공편 예약"""
    return f"{from_airport}에서 {to_airport}행 항공편 예약이 완료되었습니다."

flight_assistant = create_agent(
    model=ChatOpenAI(model="gpt-4o"),
    tools=[book_flight],
    system_prompt="당신은 항공편 예약을 도와주는 어시스턴트입니다.",
    name="flight_assistant"
)

hotel_assistant = create_agent(
    model=ChatOpenAI(model="gpt-4o"),
    tools=[book_hotel],
    system_prompt="당신은 호텔 예약을 도와주는 어시스턴트입니다.",
    name="hotel_assistant"
)
```

```
supervisor = create_supervisor(
    agents=[flight_assistant, hotel_assistant],
    model=ChatOpenAI(model="gpt-4o"),
    prompt=(
        "당신은 항공편 예약 어시스턴트와 호텔 예약 어시스턴트를 관리합니다. "
        "각 어시스턴트에게 적절히 업무를 배정하세요."
    )
).compile()
```

- 본 예시는 LangGraph의 고수준 API를 사용해 멀티 에이전트 팀을 구축한 사례.
- 개별 에이전트와 멀티에이전트의 구조와 상태를 자유롭게 변경하고 제어하기 위한 저수준 API도 지원된다는 장점이 있음.

에이전트 프로토콜

- 에이전트의 활용이 급증하면서 서로 다른 도구·에이전트 간 통신 방식의 단절 발생
 - 복잡한 실세계 문제 해결을 위해선 협업·연동이 가능한 통신 규약 필요
 - ✓ 도구 지능과 에이전트 지능의 장벽을 허물고 상호 작용하게 만들
 - ✓ 이종 에이전트의 동맹(Coalition) → 복잡한 문제 공동 해결

프로토콜	분류	주요 제공자	주요 목적	대표 활용사례
MCP	Context-Oriented	Anthropic	LLM ↔ 외부 도구 연결	검색엔진, 캘린더, 파일 등 호출
A2A	Inter-Agent	Google	에이전트 간 협업	에이전트 간 상호작용
ACP	Inter-Agent	IBM	에이전트 간 협업	에이전트 간 상호작용
Agora	Meta-Protocol	Oxford	에이전트 간 프로토콜 협상	자연어 기반 협의 및 생성
agent.json	Context-Oriented	WildCardAI	웹 API 자동 연결	브라우저 기반 웹 자동화
PXP	Human-Agent	BITS Pilani	예측·설명 중심의 전문가-에이전트 협업	방사선 진단 등
LMOS	System-Agent	Eclipse	IoA(Agents OS) 표준 인프라	에이전트 인터넷

Model Context Protocol (MCP)

- **소개** : Anthropic이 2024년 11월에 발표한 오픈 표준 프로토콜. LLM이 외부 데이터 및 도구와 **표준화된 방식으로 안전하게 상호작용**할 수 있도록 개발.
- **기능**: JSON-RPC 2.0 기반의 클라이언트-서버 구조를 갖추고 있으며, LLM이 외부의 파일, 데이터베이스, API, 슬랙, 깃허브 등과 **통일된 인터페이스**로 통신
- **장점**: 모델별/API 별로 개별 통합 코드를 작성할 필요 없이 다양한 모델들이 공통된 방식으로 외부 도구를 사용할 수 있어, **개발 효율과 확장성**이 크게 향상
- **활용 사례**: OpenAI, Google DeepMind, Microsoft Copilot Studio, Replit, Sourcegraph, Zed 등 다양한 플랫폼에서 도입

```
tools = [{
  "type": "function" ,
  "name": "get_weather" ,
  "description": "지정된 위치의 현재 온도를 가져옵니다." ,
  "parameters": {
    "type": "object" ,
    "properties": {
      "location": {
        "type": "string" ,
        "description": "도시 및 국가 예: 보고타, 콜롬비아"
      }
    },
    "required": [ "location" ],
    "additionalProperties": False
  }
}]
```

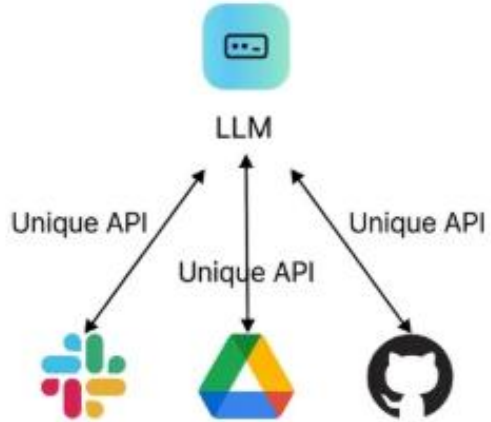
OpenAI 모델의 Tool 인터페이스

```
tools=[{
  "name": "get_weather" ,
  "description": "지정된 위치의 현재 날씨를 가져옵니다" ,
  "input_schema": {
    "type": "object" ,
    "properties": {
      "location": {
        "type": "string" ,
        "description": "도시와 주, 예: 샌프란시스코, 캘리포니아" ,
      }
    },
    "required": [ "location" ],
  },
}]
```

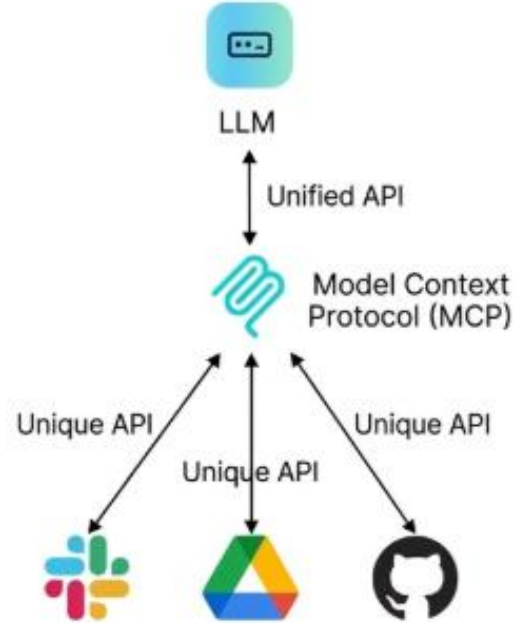
엔트로픽 모델의 Tool 인터페이스

- AI 모델과 데이터 소스/도구를 연결하기 위해서는 매번 전용 연결 코드 작성 필요
 - 문제점: 모델이 N개이고 연결해야 할 도구가 M개라면, 총 NxM개의 다른 인터페이스 구현이 필요
- MCP는 LLM ↔ 외부 도구를 연결하는 표준화된 메타데이터 교환 규격을 정의
 - MCP 서버: 표준화된 형식으로 도구 메타데이터 제공 → LLM에 종속되지 않음 (M개)
 - MCP 클라이언트/어댑터: LLM 공급자별 도구 형식에 맞춰 변환 (N개)

Before MCP

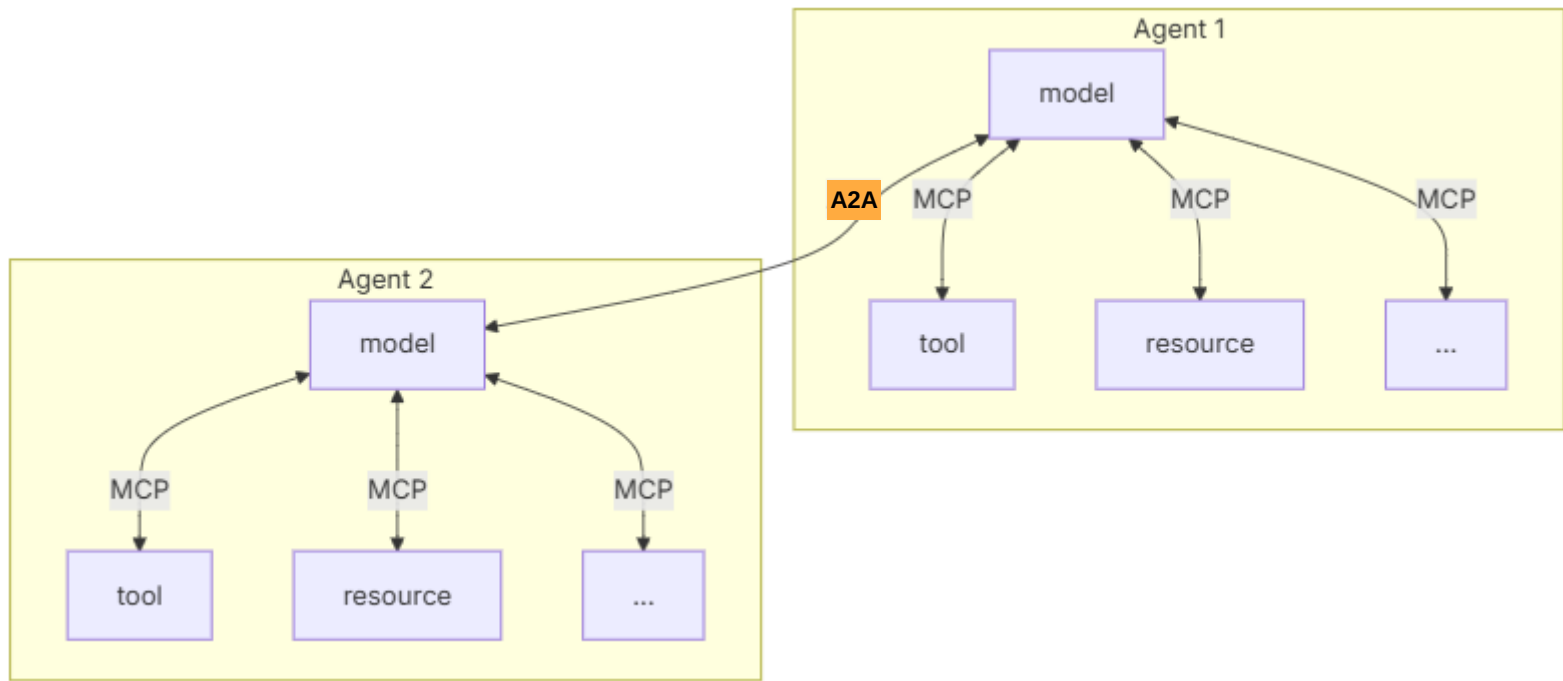


After MCP



Agent-to-Agent Protocol (A2A)

- **소개:** 2025년 구글이 발표한 오픈 프로토콜로, 서로 다른 AI 에이전트들이 서로를 **발견**, **협상**, **협업**할 수 있게 설계
- **동작 방식:** 각 에이전트는 “Agent Card”라는 JSON 메타데이터(이름, 기능, 엔드포인트, 인증 정보 등)를 공개해서 **서명 기반으로 다른 에이전트가 자신과 연결/협상/협업**하도록 작동
- **장점:** 원격의 에이전트와도 **협업과 작업 분배**를 가능하게 하며, 대규모 멀티 에이전트 시스템 구축이 가능

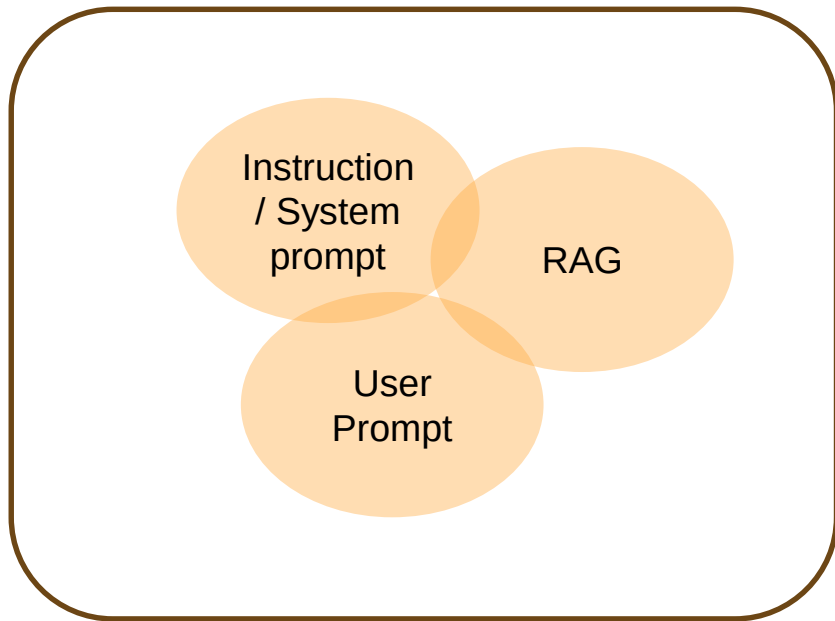


프로토콜	무엇을 해결하는가?	적합한 사용 시나리오
MCP	모델 ↔ 도구 (외부 서비스/데이터) 연결 표준화	LLM이 API, DB 등 외부 리소스에 접근해야 하는 시스템
A2A	에이전트 ↔ 에이전트 간의 협업 및 작업 위임	여러 전문 에이전트가 협력해 복잡한 문제를 해결해야 하는 구조

컨텍스트 엔지니어링

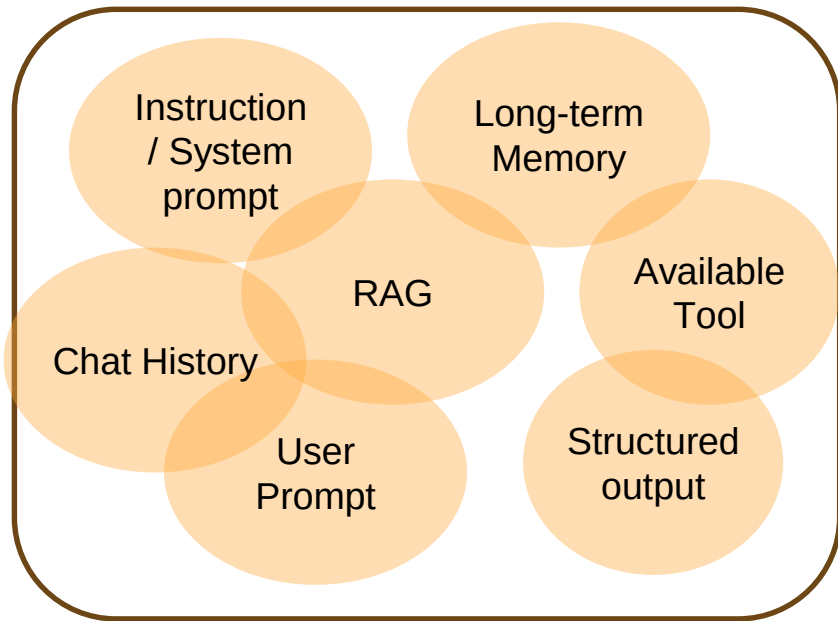
컨텍스트 엔지니어링이란?

Prompt Engineering



“모델에게 잘 요청/질문하기 위한 똑똑한 지시”

Context Engineering

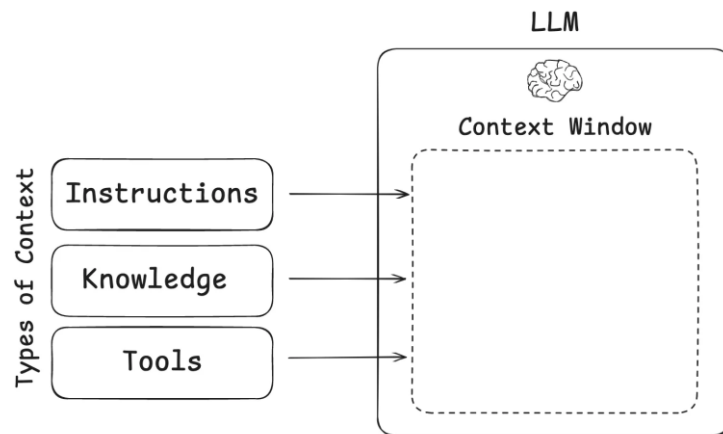
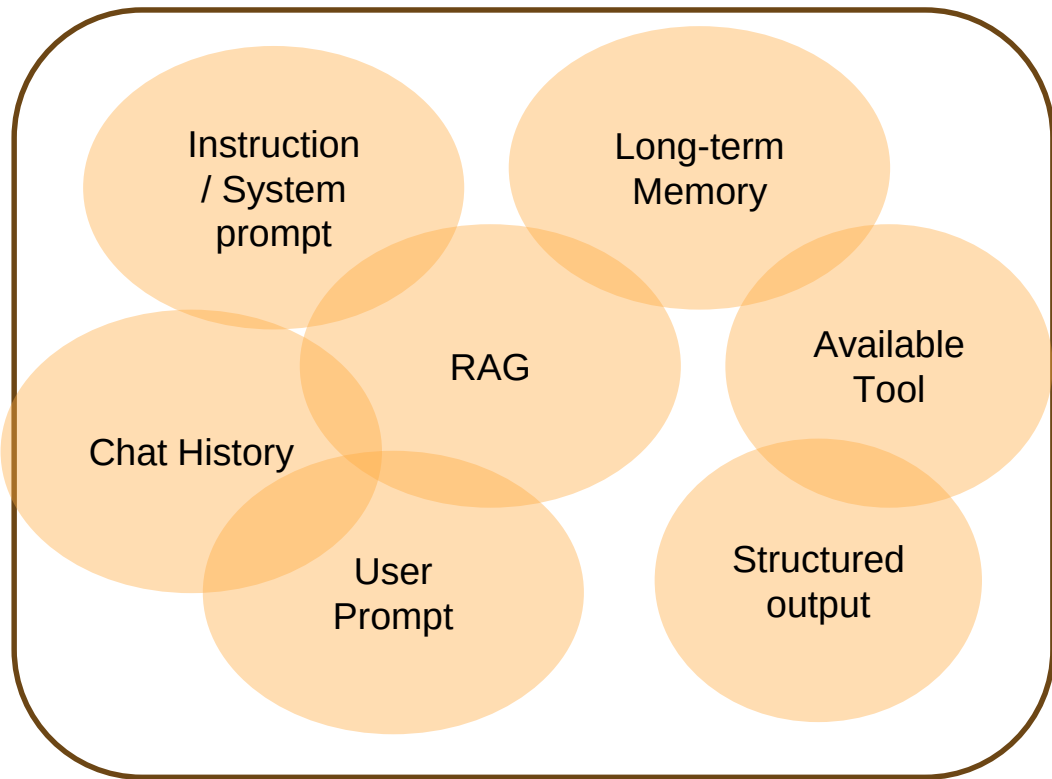


“모델이 작업을 수행하기 위해 필요한 모든 정보를 동적으로 조합”

항목	Prompt Engineering	Context Engineering
정의	모델에 전달되는 단일 입력 문장을 설계	모델이 작업을 성공적으로 수행할 수 있도록 동적으로 정보를 조합·설계하는 시스템
목적	좋은 질문으로 원하는 응답 유도	LLM이 도구와 정보를 활용하여 문제를 해결하도록 준비
범위	주로 사용자 입력 텍스트에 집중	시스템 지침, 사용자 입력, 메모리, 톨 호출, 외부 검색 등 포괄
형태	정적인 프롬프트 문장	동적으로 생성되는 다중 맥락 조합
관점	"어떻게 물어볼 것인가?"	"LLM이 작업을 완수할 수 있도록 어떤 정보들로 맥락을 구성할 것인가?"
LLM 행동 제어	행동을 프롬프트로 유도	지침 + 기억 + 도구 + 정보 = 환경 기반 행동 설계
관계	컨텍스트 엔지니어링의 하위 개념	상위 개념으로, 프롬프트 포함 전체 시스템을 설계

구분	설명
정의	LLM이 작업을 제대로 수행하도록 정보, 지침, 도구, 메모리 등을 통합하여 동적으로 맥락을 구성하는 시스템
중요성	많은 LLM 기반 시스템 실패(특히 에이전트)는 모델 성능이 아니라 맥락 제공 실패 때문→ "Garbage In, Garbage Out"
특성	◇ 시스템적 – 다양한 입력 소스(사용자, 툴, 히스토리 등)를 통합 ◇ 동적 – 프롬프트 구성 자체도 상황에 따라 유동적 ◇ 형식 중요 – 정확한 JSON 스키마, 오류 메시지, 명확한 지침이 중요
구성요소	◇ 시스템 지침 – LLM이 어떻게 행동해야 할지 명확히 규정 ◇ 유저 입력 – 실제 사용자 질의 ◇ 단기 기억 – 최근 대화 맥락 ◇ 장기 기억 – 사용자 프로필 및 과거 이력 ◇ 검색/RAG – 외부 지식 삽입 ◇ 툴 사용 – 외부 API 및 함수 호출 ◇ 커뮤니케이션 – 에이전트들 사이의 상호작용 ◇ 출력 포맷 – JSON 등 구조화된 응답 요구
실패 유형	⚠ 정보 부족 – 필요한 맥락이 없음 ⚠ 형식 오류 – 잘못된 구조, 이해 불가능한 출력 ⚠ 도구 미연결 – 작업을 완료할 능력이 없음
대표 사례	☑ 외부 API 호출을 통해 정보 검색 후 요약 ☑ 대화 요약 기반의 컨텍스트 유지 ☑ 사용자 선호 기반 추천 시스템 ☑ LLM의 출력 형식을 JSON으로 강제 ☑ LangGraph를 통한 흐름 기반 구성

Context Engineering



왜 에이전트에게 특히 중요한가?

- LLM (대형 언어 모델)은 “컨텍스트 윈도우(context window)”라는 제한된 크기의 메모리(토큰 용량)를 갖음
 - 에이전트는 특히 LLM 호출 + 도구(tool) 호출이 반복되며 여러 정보가 쌓이기 쉽다.
- 따라서 “필요한 컨텍스트만, 적절한 시점에, 적절한 형태로” LLM에게 제공하는 기술이 중요해졌으며, 이를 컨텍스트 엔지니어링이라고 지칭
- 컨텍스트 실패 모드 (Context Failure Modes)
 - 맥락 오염 (Context Poisoning) : 이전 작업의 환각이 컨텍스트 내부에 포함되어 거짓 정보가 전달됨
 - 맥락 방해 (Context Distraction) : 컨텍스트의 방대한 양에 모델이 압도당해 본래의 추론 능력을 상실
 - 맥락 혼란 (Context Confusion) : 질문과 상관없는 노이즈 컨텍스트 때문에 모델의 주의력이 분산됨
 - 맥락 충돌 (Context Clash) : 컨텍스트의 여러 부분이 서로 모순되거나 의견이 불일치할 때 발생

Write Context

Long-term memories
(across agent sessions)



Scratchpad
(within agent session)



State
(within agent session)



Select Context

Retrieve relevant tools



Retrieve from scratchpad



Retrieve long-term memory



Retrieve relevant knowledge



Compress Context

Summarize context
to retain relevant tokens



Trim context to
remove irrelevant tokens



Isolate Context

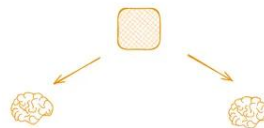
Partition context in state



Hold in environment/sandbox



Partition across multi-agent



 = LLM Context

작성 (Write Context)

- **개념:** 컨텍스트를 작성한다는 것은 에이전트가 작업을 수행하는 데 도움이 되도록 컨텍스트 창 외부에 컨텍스트를 저장하는 것을 의미
- 주요 기법:
 - **Scratchpad (스크래치패드):**
 - ✓ 단일 세션 작업 중 중간 산출물, 계산 결과, 메모 등을 기록해 두는 공간이다.
 - ✓ 예: 에이전트가 문제를 풀면서 중간 생각을 메모장처럼 파일에 적어두고, 나중에 참조하거나 이어서 사용한다.
 - **Memory (기억 / 장기 저장소)**
 - ✓ 여러 세션을 넘어 정보를 보관하는 방식이다.
 - ✓ 예: 어떤 사용자의 취향, 이전 대화 내용 요약, 규칙들 등은 매번 새로 제공하기보다 기억 공간에 저장해 두고 필요할 때 꺼내 쓰는 식.

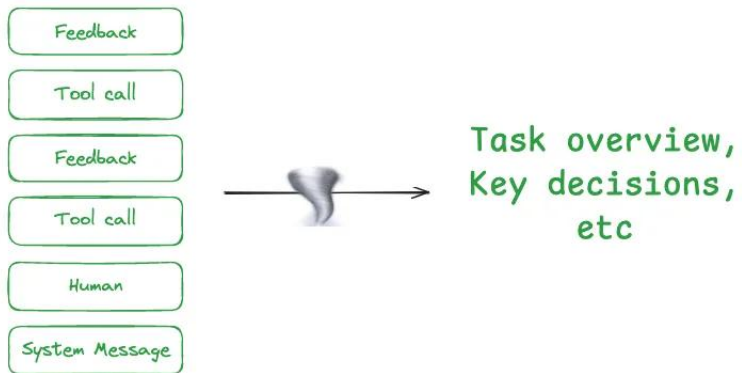
선택 (Select Context)

- **개념:** 매 순간 작업에 **가장 관련성 높은 정보**를 방대한 데이터 소스(DB, 문서, 웹)에서 '선별하여' 가져오는 동적인 방식입니다. RAG(검색 증강 생성)가 대표적
- 주요 기법:
 - **메모리 검색**
 - ✓ 작업과 관련된 관련 메모리를 선택적으로 회상
 - ✓ 절차, 사실, 에피소드 등 해당 기억에 적절한 메모리 스키마로 다루는 것이 중요
 - **도구 선정**
 - ✓ 에이전트가 너무 많은 도구를 제공받아 혼란을 겪는 것을 방지하기 위해 사용
 - ✓ RAG(검색 증강 생성)를 도구 설명에 적용하여 작업에 가장 관련성 높은 도구만 가져옴
 - **스크래치패드에서 선택적으로 꺼내기**
 - ✓ 스크래치패드가 파일 또는 상태 객체 형태라면, 그 중 일부 항목만 읽어서 LLM에게 노출

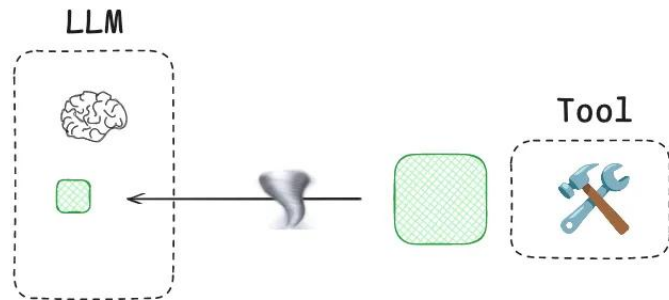
압축 (Compress Context)

- **개념:** '선택'된 정보가 너무 길거나 장황하여 LLM의 컨텍스트 창(Context Window)에 모두 담기 어려울 때, **핵심만 남기고 줄이는** 과정
- 주요 기법:
 - **컨텍스트 요약 (Context Summarization)**
 - ✓ 수백 턴에 걸친 에이전트 상호작용이나 토큰 사용량이 많은 도구 호출의 피드백을 관리
 - **컨텍스트 트리밍 (Context Trimming)**
 - ✓ 단순히 오래된 메시지, 중요도 낮은 대화, 중복 내용 등을 제거하는 방식.

Summarize message history

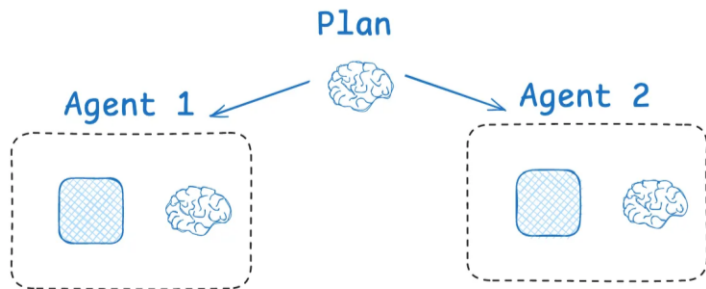


Summarize tool feedback



분리 (Isolate Context)

- **개념:** 복잡한 작업을 여러 개의 단순한 하위 작업으로 나눈 뒤, 각 하위 작업을 처리할 때 다른 정보로부터 방해받지 않도록 '격리된 작업 공간'을 만들어주는 전략
- 구현 방식:
 - 하나의 '마스터 에이전트'가 전체 계획을 세움
 - 각 하위 작업은 오직 그 작업에 필요한 도구와 정보만 가진 '전문가 에이전트'에게 할당
 - 이 전문가 에이전트는 다른 작업의 컨텍스트를 볼 수 없으므로, 자신의 임무에만 집중



Parallelize computation with isolated context windows

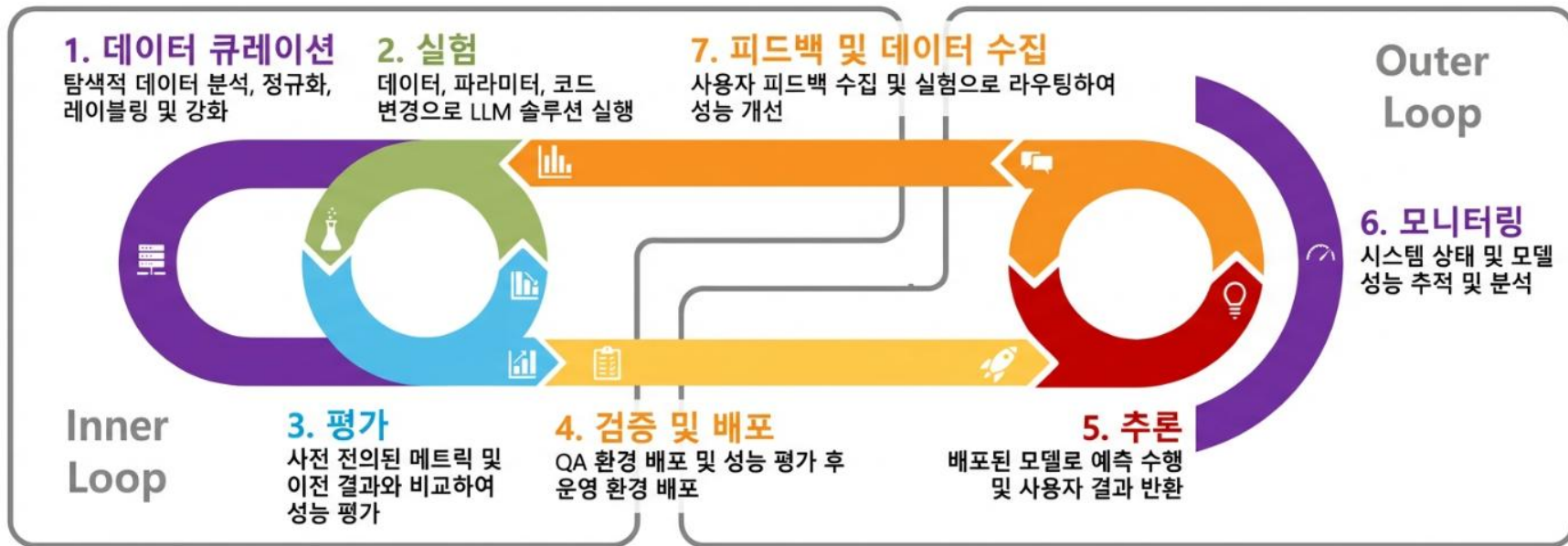
전략	핵심 아이디어	언제 쓸까	장점	주의점
Write Context	외부 저장소에 정보 기록	누적 정보, 반복 정보 관리할 때	토큰 낭비 방지, 정보 보존	저장된 정보가 많아지면 선택이 어려워짐
Select Context	필요한 정보만 꺼내 노출	저장된 정보이 많을 때	불필요한 정보 제거, 집중도 향상	선택 오류 → 중요한 정보 누락 위험
Compress Context	요약 또는 가지치기로 축소	대화 로그 길어질 때	토큰 절감, 핵심 집중	과도 요약 시 정보 손실 가능
Isolate Context	여러 분리된 컨텍스트로 분할	복잡한 작업, 멀티 에이전트 설계 시	충돌 감소, 역할 분리	분리 간 조정 / 병합 구조 설계 필요

에이전트 운영 및 평가

프로덕션 환경을 위한 실용적인 과제

- 효과적인 평가 (Effective Evaluation)
 - 문제: 에이전트의 성능을 일관된 기준으로 측정하기 힘들.
 - 해결책: LLM을 심판으로 사용(LLM-as-a-judge), 인간 참여 (Human-in-the-loop)
- 디버깅 및 관찰 가능성 (Debugging & Observability)
 - 문제: 에이전트의 의사결정 과정은 '블랙박스'와 같아 문제 원인 파악이 어려움.
 - 해결책: 에이전트의 모든 생각, 도구 사용, 중간 결과를 추적하고 시각화하는 **관찰 도구(e.g., LangSmith)**가 필요.
- 개선 및 재배포 (Improvement & Deployment)
 - 문제: 에이전트의 성능을 지속적으로 개선하고 안정적으로 운영 환경에 반영하기 어려움.
 - 해결책: 오프라인 평가 + 온라인 A/B 테스트를 통한 성능 검증, CI/CD 기반 자동 배포, 실시간 모니터링 및 피드백 루프 구축

LLMOps Lifecycle



에이전트 평가 전략

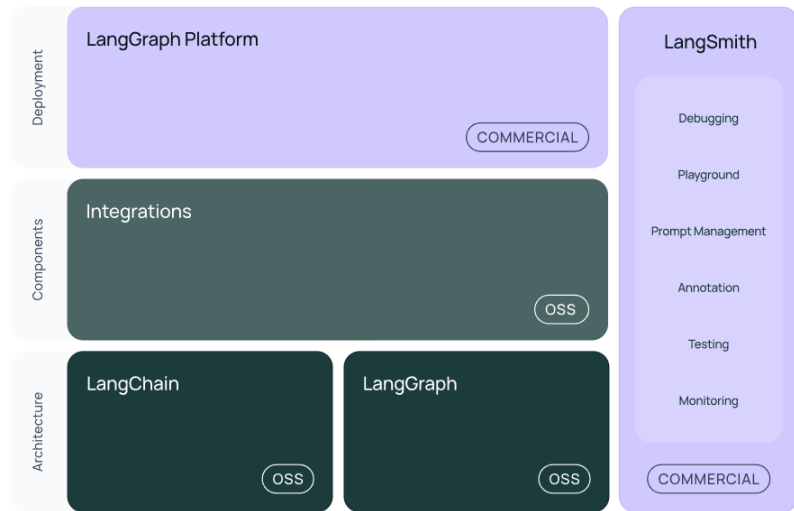
- 전략 1: 최종 응답 평가 (Final Response Evaluation)
 - 초점: 에이전트의 최종 출력이 사용자의 문제를 해결했는가?
- 전략 2: 단일 단계 평가 (Single-Step Evaluation)
 - 초점: 워크플로우 내 특정 결정(예: 올바른 도구 선택)이 정확했는가?
- 전략 3: 궤적 평가 (Trajectory Evaluation)
 - 초점: 최종 결론에 도달하기까지의 과정(추론, 도구 호출 순서)이 논리적이고 효율적이었는가?

LLM-as-Judge

- 개념: 하나의 LLM을 사용하여 다른 LLM(또는 에이전트)의 출력을 평가하는 기법.
 - 장점: 복잡하고 미묘한 기준(예: 유용성, 어조)에 대해 확장 가능하고 자동화된 평가 가능.
- 구현 5단계:
 - 시나리오 정의: 무엇을 평가할지 명확히 함 (정확성, 어조 등).
 - 데이터셋 준비: 평가할 예시 데이터 수집.
 - 수동 레이블링: '정답(Ground Truth)' 데이터셋 구축.
 - 평가자 프롬프트 작성: 평가 기준, 채점 기준표, 출력 형식을 명시하는 것이 핵심.
 - 평가 및 반복: 심사자의 출력을 정답과 비교하며 프롬프트 개선.

왜 관찰 가능성이 중요한가?

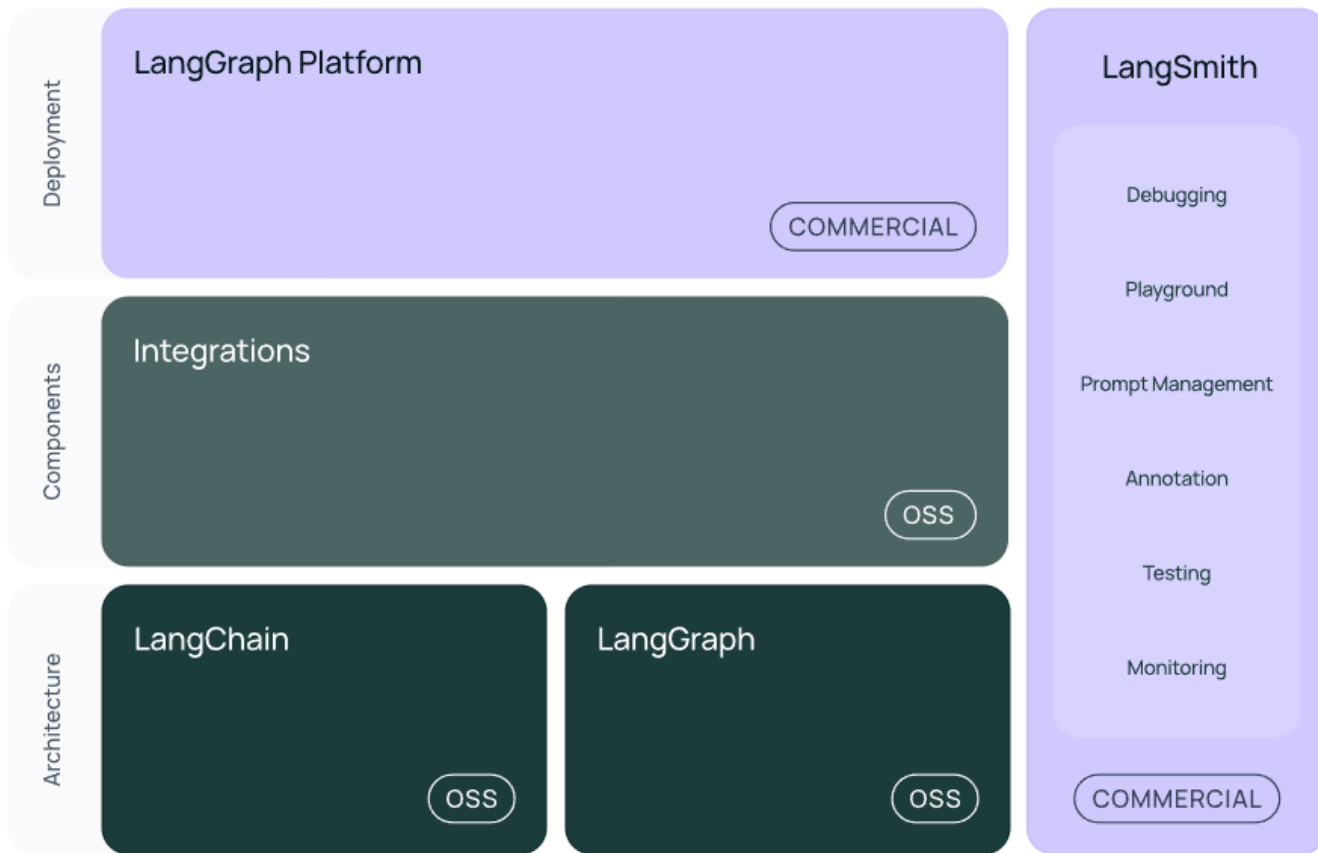
- "블랙박스" 문제: 에이전트는 비결정적 특성으로 인해 행동 예측 및 디버깅이 어려움.
- 기존 도구의 한계:
 - 단순 로그는 에이전트의 복잡한 "사고 과정" 포착 실패
 - (LLM 호출, 도구 입출력, 내부 추론 단계의 전체 순서)
- LangSmith:
 - LLM 애플리케이션 수명 주기를 위해 특별히 설계
 - 트레이싱, 디버깅, 모니터링, 평가 도구를 제공



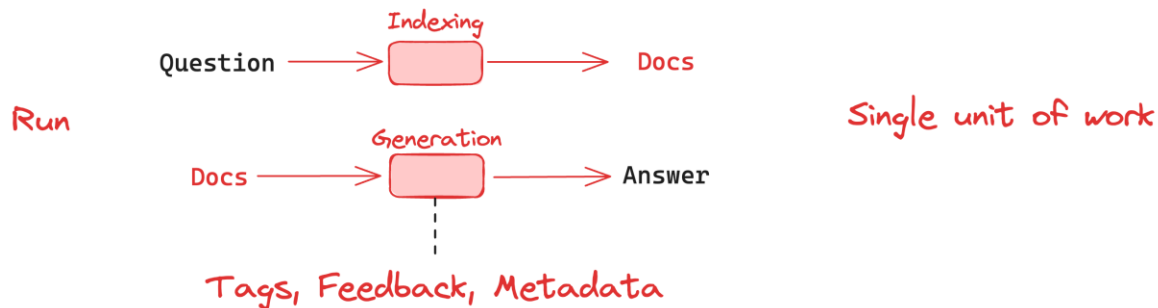
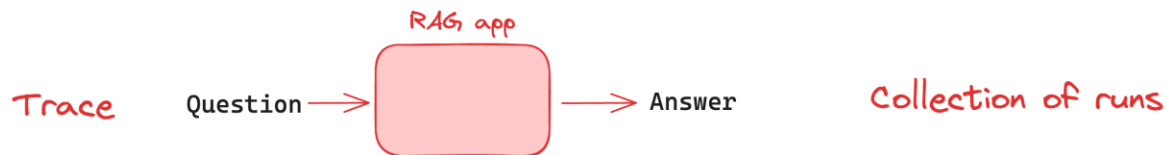
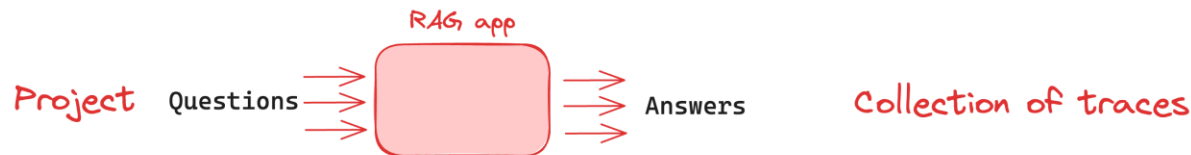
LangSmith 소개

- LLM 애플리케이션 수명 주기를 위해 설계된 통합 플랫폼.
 - *“Building production-grade LLM applications”* - LangSmith
- LangChain Native
 - LangChain 프레임워크와 매우 간단한 코드로 연동 가능
 - 별도의 복잡한 설정 없이 모든 Chain과 Agent 내부를 자동 추적(Trace)
 - LLM 앱을 운영하기 위한 디버깅, 모니터링, 평가 도구를 제공





LangSmith's Observability Concepts



LangChain Inc. > Projects > chat-langchain

chat-langchain

See: https://chat.langchain.com/

TracesLLM CallsMonitorSetup

Last 7 days

eg. eq(run_type, "chain")

>	👤	Name	Input	Start Time	Latency	Dataset	Annotation Queue	Tokens	Cost	Fi
>	👤	/chat		2/13/2024, 12:29:01...				478	\$0.000727	10
>	👤	/chat	langchain-cli 主要作用...	2/13/2024, 12:28:45...	🕒 3.20s			3,146	\$0.0047585	25
>	👤	/chat	数据分析师如何做?	2/13/2024, 12:28:38...	🕒 12.23s			3,067		42
>	👤	/chat	peux-tu me synthétise...	2/13/2024, 12:28:35...	🕒 4.12s			4,166	\$0.0062695	22
>	👤	/chat	peux-tu me synthétise...	2/13/2024, 12:28:03...	🕒 1.85s			6,082	\$0.0091505	12
>	👤	/chat	Explain multi-vector re...	2/13/2024, 12:27:40...	🕒 7.19s			4,175		71
>	👤	/chat	I have these set into m...	2/13/2024, 12:26:58...	🕒 3.87s			7,147	\$0.0108	55
>	👤	/chat	how to use directorylo...	2/13/2024, 12:26:10...	🕒 4.28s			5,189		25
>	👤	/chat	output import sentenc...	2/13/2024, 12:25:45...	🕒 4.30s			5,656	\$0.008496	18
>	👤	/chat	File "C:\Users\rahul.pe...	2/13/2024, 12:25:43...	🕒 2.98s			4,464		47
>	👤	/chat	how to modify in a way...	2/13/2024, 12:25:38...	🕒 6.94s			7,028	\$0.010624	15
>	👤	/chat	ValueError: Expected ...	2/13/2024, 12:25:32...	🕒 1.09s			4,074		85
>	👤	/chat	how do i get a function...	2/13/2024, 12:25:28...	🕒 4.60s			2,518	\$0.0038745	17
>	👤	/chat	same for functions? su...	2/13/2024, 12:25:02...	🕒 6.45s			5,561	\$0.0084185	18

Load more

Details

RUN COUNT

26,434

TOTAL TOKENS

204,328,819 / \$95.2410105

MEDIAN TOKENS

5,907

ERROR RATE

1%

% STREAMING

99%

LATENCY

P50: 6.47s P99: 55.20s

FIRST TOKEN

P50: 1.49s P99: 10.36s

FEEDBACK

ALWAYS_PASS : 1 CORRECTNESS : 1

FAITHFULNESS_SCORE : 0.5

GRAMMATIC_CORRECTNESS-OPTIMALITY-HALLUCINATIONS : 0

JRCP : 42

WOW WHAT DUDE : -4

USER_SCORE : 0.35

USER_CLICK : 0

Project (프로젝트):

애플리케이션의 성격이나 환경에 따라 **트레이스 (Trace)**들을 그룹화하여 관리하는 최상위 작업 공간

The screenshot displays the LangChain interface for a project named 'chat-langchain'. The main view shows a 'Trace' for a 'Retriever' step, which is highlighted with a red box. The trace details include the input query, the documents retrieved, and the output generated by the LLM.

Trace Details:

- Input:** A JSON object with a 'query' field: `{ "query": "How do I use a RecursiveUrlLoader to load content from a page?" }`
- Output:** A list of documents retrieved, including 'Recursive URL', 'langchain_community.document_loaders.recursive_url_loader.RecursiveUrlLoader', 'lazy_load()', 'URL', 'WebBaseLoader', and 'langchain_community.document_loaders.news.NewsURLLoader'.
- Status:** Success
- Latency:** 0.37s
- Tags:** Weaviate, seqstep:2, model:openai_gpt_3_5_turbo, OpenAIEmbeddings

Trace (트레이스):

사용자의 질문(Input)부터 최종 답변(Output)까지 이어지는 하나의 완전한 실행 흐름입니다.

사용자 요청(Trigger)으로 시작된 체인 실행, LLM 호출, 결과 파싱 등 모든 일련의 하위 과정을 하나로 묶은 전체 실행 기록(Full Execution Path)

Trace Collapse Stats Most relevant

/chat @ SUCCESS

3.63s 4.127 model:openai_gpt_3_5_turbo

Retriever 0.37s

ChatOpenAI 3.14s

Some runs have been hidden. [Show 15 hidden runs](#)

Retriever

Run Feedback Metadata

Input

```
1 {
2   "query": "How do I use a RecursiveUrlLoader to load content from a page?"
3 }
```

JSON

Output

DOCUMENTS

- Recursive URL | Langchain [Skip to main content](#_docsaus... Recursive URL | Langchain +1
- langchain_community.document_loaders.recursive_url_loader.RecursiveUrlLoader ~ La... +2
- lazy_load() Lazy load web pages. load() Load web pages. load_and_split(text_splitter) Load D... +2
- URL | Langchain [Skip to main content](#_docsausurus_skipToConten... URL | Langchain +1
- WebBaseLoader | Langchain [Skip to main content](#_docu... WebBaseLoader | Langchain +1
- langchain_community.document_loaders.news.NewsURLLoader ~ LangChain 0.1.4 Lang... +2

START TIME
02/13/2024, 12:15:14 AM

END TIME
02/13/2024, 12:15:14 AM

TIME TO FIRST TOKEN
N/A

STATUS
Success

TOTAL TOKENS
0 tokens

LATENCY
0.37s

TYPE
Retriever

TAGS
Weaviate seqstep:2
model:openai_gpt_3_5_turbo
OpenAIEmbeddings

Run (런):

트레이스 내부를 구성하는 LLM 호출, 검색, 도구 사용 등 각각의 세부 실행 단위

LangChain Inc. > Projects > chat-langchain > /chat > ... > ChatOpenAI

Trace [Collapse] [Stats] [Most relevant]

/chat SUCCESS

3.63s 4.127 model:openai_gpt_3_5_turbo

Retriever 0.37s

ChatOpenAI 3.14s

Some runs have been hidden. [Show 15 hidden runs](#)

ChatOpenAI

Run Feedback Metadata

Created by	Key	Score	Comment	Correction	Source	Source Info
A	CORRECTNESS	1			APP	

< > Show 10

피드백(Feedback)은 개별 실행(Run)을 평가하는 기능

각 피드백 항목은

- feedback tag
- feedback score

로 구성되며, **Run ID**로 해당 실행과 연결

LangSmith 대시보드

- 쌓인 데이터를 대시보드에서 거시적으로 모니터링 가능
 - 대시보드를 통해 핵심 메트릭(지연 시간, 비용, 오류율, 품질 점수) 시각화
 - ✓ Run Count: 총 호출 횟수 (트래픽 추이)
 - ✓ Error Rate: 전체 호출 중 실패 비율 (안정성)
 - ✓ Latency (P50/P99): 평균 및 최악 응답 속도
 - ✓ Token Usage: 비용과 직결되는 총 토큰 소비량
 - 경고 설정: 특정 임계값 초과 시 자동 알림 (예: P99 지연 시간이 10초 초과)
- 사용자 피드백:
 - 애플리케이션 품질에 대한 가장 중요한 지표.
 - UI 또는 SDK를 통해 피드백을 수집하고, 이를 평가 데이터셋으로 활용.

kehuzhichijiqiren

Prebuilt Dashboard for kehuzhichijiqiren

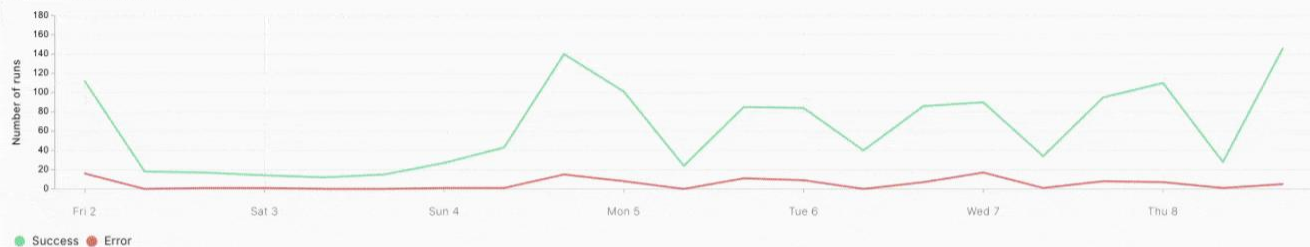
Traces LLM Calls Cost & Tokens Tools Run Types Feedback Scores

Group by Last 7 days

Traces

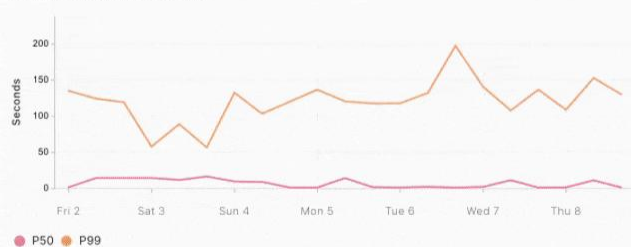
Trace Count

Total number of traces over time



Trace Latency

Trace latency percentiles over time



Trace Error Rate

Percent of traces that errored over time



LLM Calls

LLM Count

Number of LLM calls over time



LLM Latency

LLM call latency percentiles over time



대시보드를 통한 지표 추적 기능 (토큰 사용량)

Name

token usage

Description

SELECT TRACING PROJECTS AND FILTER RUNS

Tracing project(s) ⓘ

chat-langchain-v2 ✕ Select projects...

Required

Filtered on ⓘ

≡ 1 filter

PICK A METRIC

Choose metric ⓘ

Prompt Tokens

Required

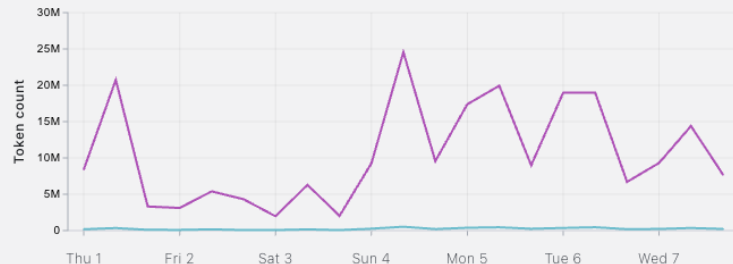
Compare to ⓘ

Completion Tokens

Optional

⌵ Last 7 days

token usage



☐	☒	Name	Input	Output	Error	Start Time
☐	☒	RetrievalGraph	user: yes	Great! Here's a co...		5/8/2025, 5:34:51 PM
☐	☒	RetrievalGraph	user: is m...	Based on the avail...		5/8/2025, 5:34:21 PM
☐	☒	RetrievalGraph	user: Sam...	Here is a sample R...		5/8/2025, 5:33:36 ...
☐	☒	RetrievalGraph	user: is th...	No, the MKL (Math...		5/8/2025, 5:32:28 P...
☐	☒	RetrievalGraph	user: yes	Excellent! Here's h...		5/8/2025, 5:31:04 PM
☐	☒	RetrievalGraph	user: how ...	Based on the sear...		5/8/2025, 5:30:53 P...
☐	☒	RetrievalGraph	user: ves	Great! Let's extend...		5/8/2025, 5:29:53 P...

메타데이터 기반 그룹화 및 성능 비교 분석

» Edit Chart

Create data series ⓘ

openai



(x) Metadata is

Where model_name is



openai_gpt_4o_mini

+ Add filter

[Advanced filters](#)

anthropic



(x) Metadata is

Where model_name is



anthropic_claude_3_haiku

+ Add filter

[Advanced filters](#)

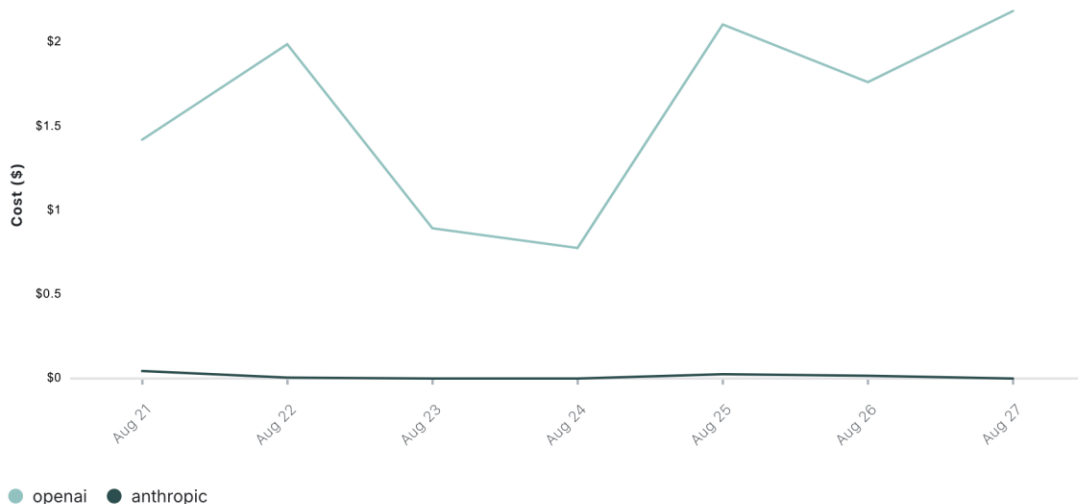
+ Data series

Last 7 days

Total cost

The total cost across all projects

모델에 따른 총비용 분석

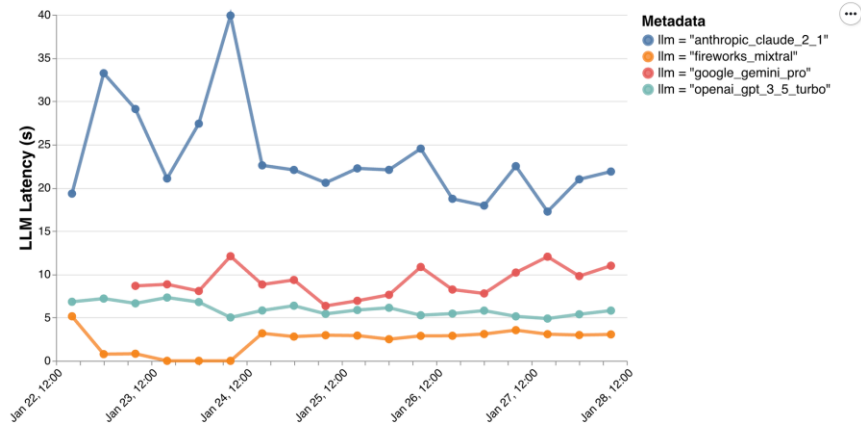


chat-langchain-langgraph-...

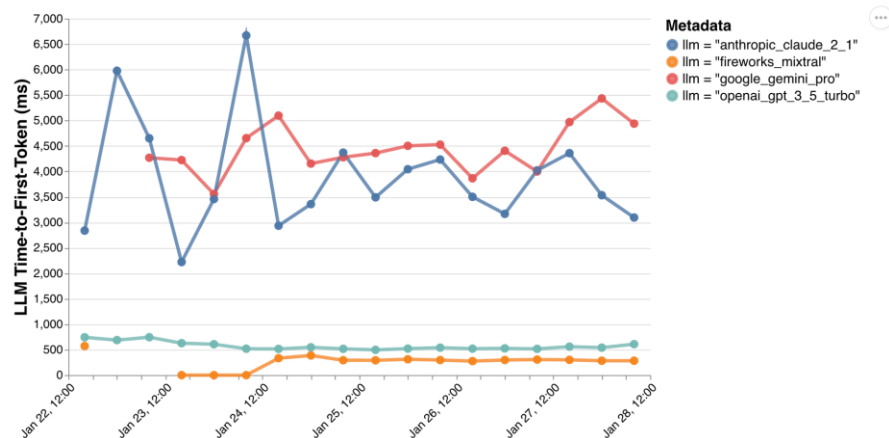
evaluators

playground

모델에 따른 지연시간



모델에 따른 TTFT(첫 토큰 도달 시간)



- Data Segmentation (데이터 그룹화)
 - 동일한 태그(Tag)나 메타데이터(Metadata)를 공유하는 트래이스들을 그룹핑하여 통계 산출
 - ✓ 예시: model: gpt-4 그룹 vs model: gpt-3.5 그룹, prompt_version: v1 vs v2
- Comparative Metrics (다차원 성능 비교)
 - 그룹 간의 핵심 성과 지표를 시각적으로 대조하여 Trade-off 분석. Latency, Cost, Quality 등
- A/B Testing & Decision Making
 - "새로운 프롬프트가 비용은 10% 줄였지만, 품질(피드백)은 유지했는가?" 검증

예시) 특정 노드에서 이메일에 대한 결정을 내리는 방식 분석

» Edit Chart

Required

Create data series ①

no response

Where (K) Output KV is triage.response is no

+ Add filter

[Advanced filters](#)

email

Where (K) Output KV is triage.response is email

+ Add filter

[Advanced filters](#)

notify

Where (K) Output KV is triage.response is notify

+ Add filter

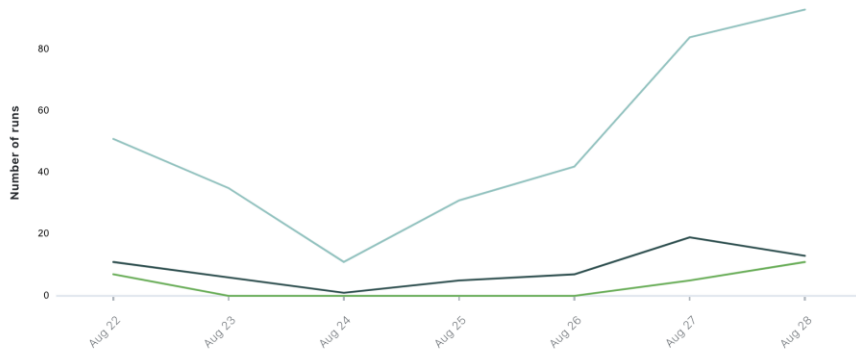
[Advanced filters](#)

Cancel

Save

Last 7 days

route decisions



no response email notify

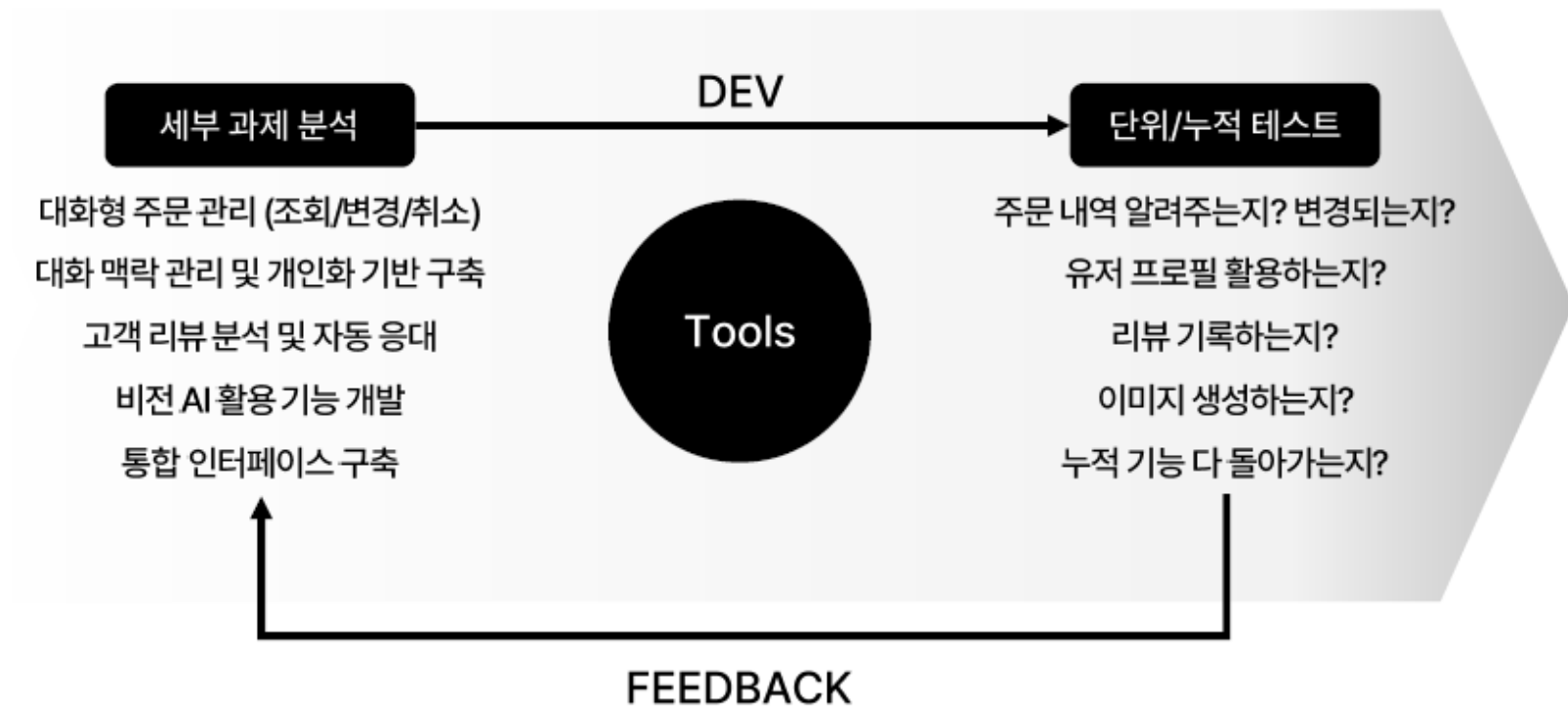
email-assistant-v2

no response email notify

☐	☒	Name	Input	Output	Start Time	Latency
☐	☒	support	{"id":"1919efe7b3d72..."}	{"email":{"id":"19..."}}	8/29/2024, 9:45:22...	2.26s
☐	☒	support	{"id":"1919f04004d5c..."}	{"email":{"id":"19..."}}	8/29/2024, 9:45:19 ...	1.56s
☐	☒	support	{"id":"1919ef3283e60..."}	{"email":{"id":"19..."}}	8/29/2024, 9:32:26...	1.77s

에이전트 구축을 위한 절차

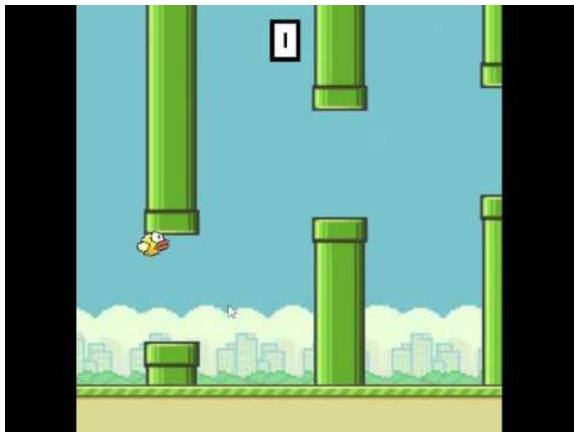
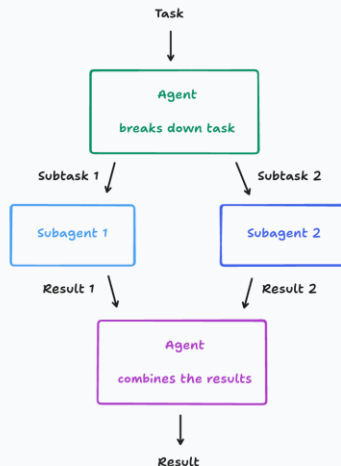
단계	Define	Design Standard Operating Procedure	Build MVP for Core Prompts	Connect & Orchestrate	Test & Iterate	Deploy, Scale & Refine
설명	현실적인 에이전트 사용 사례를 식별하고 에이전트가 처리해야 할 5-10개의 구체적인 시나리오를 수집합니다.	작업을 사람이 따라 명확한 단계로 나누어 단계별 절차를 만듭니다.	에이전트 아키텍처를 설계하고 핵심 추론을 처리하는 프롬프트를 구축합니다.	실시간 데이터 소스에 연결하고 오케스트레이션 로직을 구축합니다.	예시를 바탕으로 에이전트를 평가하고, 실패 사례를 식별하여 신뢰성과 성능을 개선하기 위해 반복합니다.	에이전트를 배포하고 사용자의 상호작용을 모니터링하며, 실제 사용 패턴을 기반으로 지속적으로 개선합니다.
기간	~1-2 주	~1-2 주	~1-2 주	~2-3 주	~1-2 개월	지속적
결과물	명확한 에이전트 작업 범위 및 구체적인 사용 사례	식별된 시나리오를 통합한 단계별 워크플로우의 명확한 표준 운영 절차	정적 데이터로 작동하는 에이전트 아키텍처 및 핵심 프롬프트	실시간 데이터가 연동된 엔드투엔드(End-to-end) 에이전트	체계적으로 성능이 테스트된 에이전트	기능이 확장되고 개선된 에이전트
이해관계자	제품 책임자, 도메인 전문가	제품 관리자, 도메인 전문가	제품 관리자, AI 및 프롬프트 엔지니어	엔지니어	제품 관리자, QA 엔지니어	PM, 엔지니어, 지원팀
예시: 이메일 에이전트 구축	- 우선순위에 따라 이메일 분류 - 회의 일정 조율 - 제품 관련 질문에 답변	단계별 절차: - 수신 이메일에 대해 카테고리 및 응답 우선순위 라벨링 - 캘린더 가용성 확인 및 회의 일정 조율 - 응답 초안 작성 및 사람의 검토를 위해 대기열에 추가	- 이메일 라우팅 및 응답 생성을 포함한 핵심 프롬프트로 이메일 에이전트 아키텍처 설계. - 수동으로 제공된 컨텍스트와 샘플 데이터로 테스트.	통합 예시: - 이메일 접근을 위한 Gmail 및 Outlook - 구글 및 Outlook 캘린더 - 발신자 컨텍스트를 위한 CRM 데이터베이스	성공 기준에 따른 이메일 응답 평가: 응답 품질, 정확성, 전문적인 톤	- 트래픽, 사용 사례, 피드백을 모니터링. - 성능을 개선하고 추가 기능을 더하기 위해 반복.



Apendix.멀티 에이전트 담론

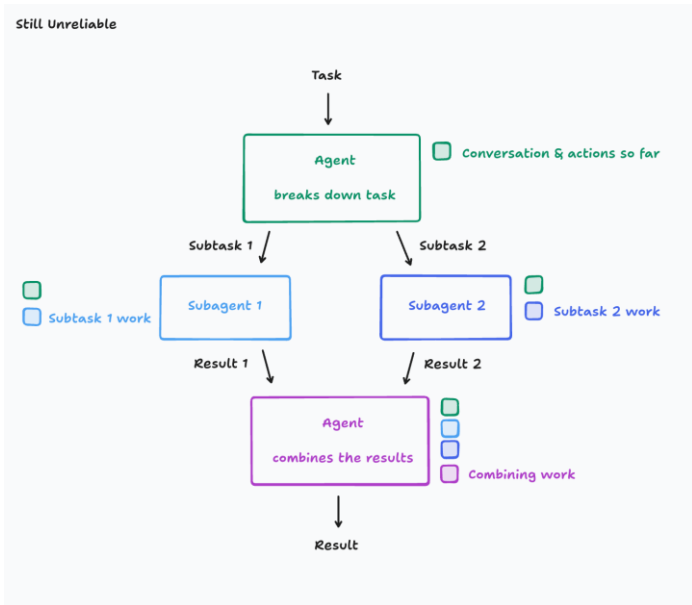
Cognition의 주장: Don't Build Multi-Agents!

- 멀티 에이전트를 구축하지 마라!
- 멀티 에이전트 시스템이 실패하는 여러 가지 이유
 - 분산된 의사결정으로 인한 충돌
 - 부실한 컨텍스트 공유 (Poor Context Sharing)로 인한 불일치

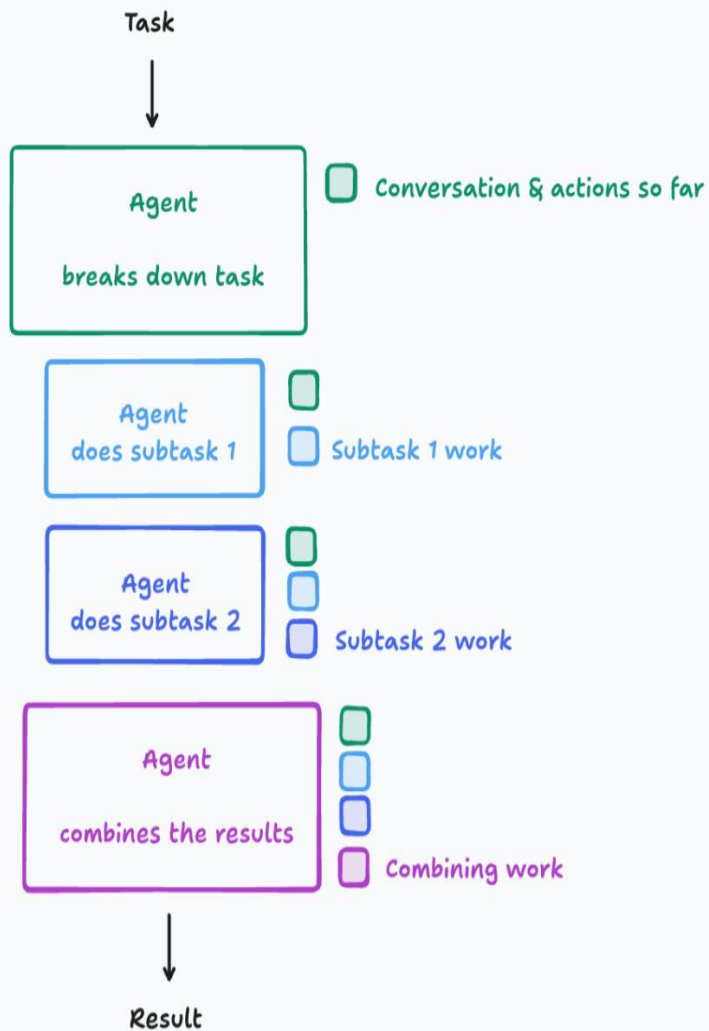


• 순진한 멀티 에이전트: 분업의 함정

- **문제 상황:** 플래너가 작업을 분해하고, 서브 에이전트가 순서대로 수행
 - ✓ 에이전트 1: '새'의 물리 로직(점프, 중력)만 구현.
 - ✓ 에이전트 2: 이후 '파이프 장애물' 로직을 추가하려 함.
- **실패 원인:** 컨텍스트 단절 (Context Silo)
 - ✓ 에이전트 1은 '파이프'의 존재를 전혀 모른 채 코드를 작성.
 - ✓ 결과적으로, 나중에 파이프를 추가하거나 상호작용하기 어려운 구조의 코드가 생성됨.
- **결론**
 - ✓ 각 에이전트는 맡은 임무를 성공했으나, 결합이 불가능해 전체 프로젝트는 실패.
 - ✓ 단순한 작업 분담은 컨텍스트를 파편화시켜 통합할 수 없는 결과를 낳는다.



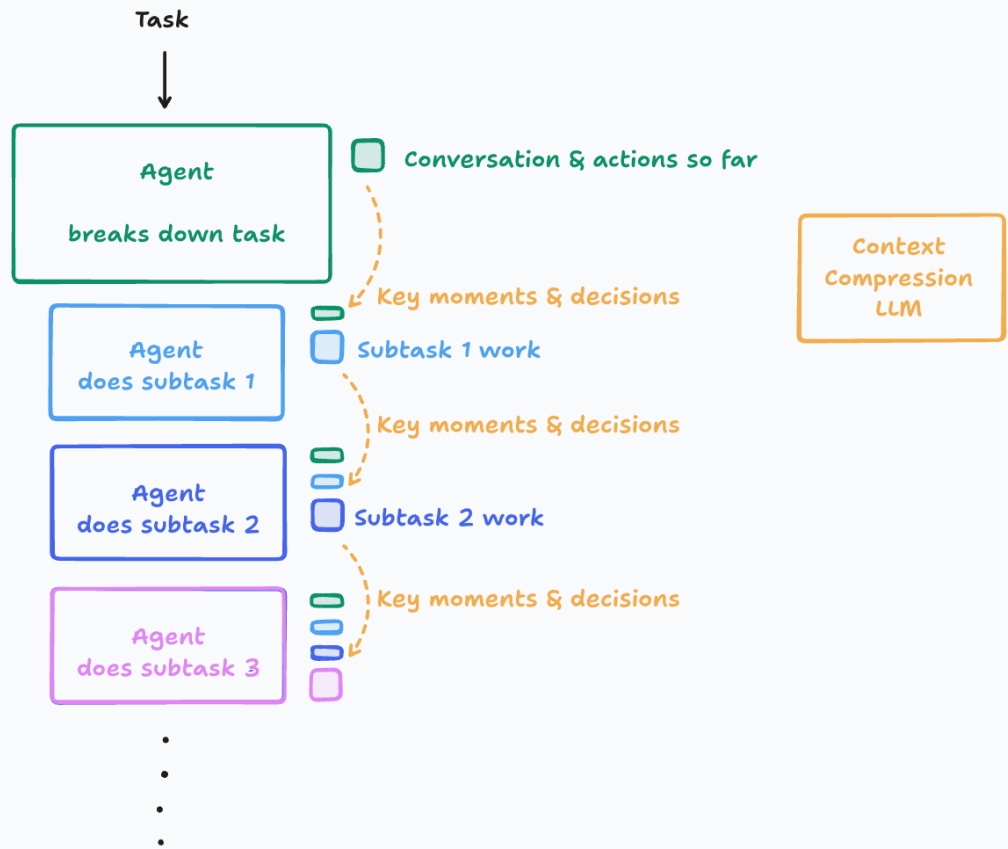
- 컨텍스트 공유 시도와 새로운 문제들
 - 제안 1: 전체 계획(Full Plan) 공유
 - ✓ '무엇을' 할지는 알지만, 각 요소가 '어떻게' 상호작용해야 하는지에 대한 깊은 이해는 여전히 부재.
 - 제안 2: 전체 파일(Full File) 공유
 - ✓ 의도 소실: 코드 자체만으론 '왜' 그렇게 작성됐는지 파악 불가.
 - ✓ 비용 폭증: 작은 수정 하나를 위해 매번 전체 코드를 분석하는 것은 극심한 비효율.
 - 결론
 - ✓ 컨텍스트를 억지로 주입하려는 시도는 근본적인 해결책이 아니며, 또 다른 복잡성과 비효율을 낳는다.



- 대안: 단일 에이전트 (Monolithic Agent)

- 작업을 나누지 않고, 하나의 에이전트가 모든 컨텍스트를 보유한 채 전체를 책임
- **이유**
 - ✓ **완벽한 컨텍스트**: 외부와 공유할 필요 없이, 에이전트 내부에서 모든 정보가 완벽히 보존
 - ✓ **설계의 일관성**: 계획부터 실행까지 하나의 주체가 담당하여 일관된 아키텍처 유지.
 - ✓ **구조적 단순함**: 에이전트 간의 복잡한 통신과 동기화 문제가 사라져 시스템이 견고
- **결론**
 - ✓ 어설픈 협업보다, 모든 것을 아는 하나의 전문가가 더 나은 결과를 만든다.

Reliable on longer tasks
(but hard to get right)



Anthropic의 반론

- 다중 에이전트가 효과적인 분야가 있다!
- 다중 에이전트 기반 연구의 진화
 - 왜 기존 AI는 연구에 어려움을 겪었는가?
 - ✓ 예측 불가능성: 연구는 정해진 경로가 없는 '개방적 탐색' 과정
 - ✓ 경직된 파이프라인: 기존의 단일 모델, 선형적 접근 방식은 유연성이 부족
 - 분산 인지를 통한 도약
 - ✓ 단일 지능체에서 분산된 전문 에이전트들의 시스템으로 전환하는 개념적 도약
 - ✓ 하나의 복잡한 문제를 여러 개의 하위 문제로 분해하고, 각 하위 문제를 전문 에이전트에게 할당함으로써 인지적 부담을 분산
 - ✓ 이런 구조는 AI 시스템에 전례 없는 수준의 자율성과 적응성을 부여

해법: 분산된 지능, '오케스트레이터-워커' 모델

- **핵심 아이디어:**
 - 하나의 복잡한 문제를 여러 개의 하위 문제로 분해
- **리드 에이전트 (오케스트레이터):**
 - 연구팀의 '팀장' 역할 (Claude Opus 4).
 - 전체 전략 수립, 하위 에이전트 생성 및 관리
- **하위 에이전트 (워커):**
 - '전문 연구원' 역할 (Claude Sonnet 4).
 - 명확히 정의된 임무를 병렬적으로 수행하고 결과 보고

Claude.ai chat



What's on your mind tonight?

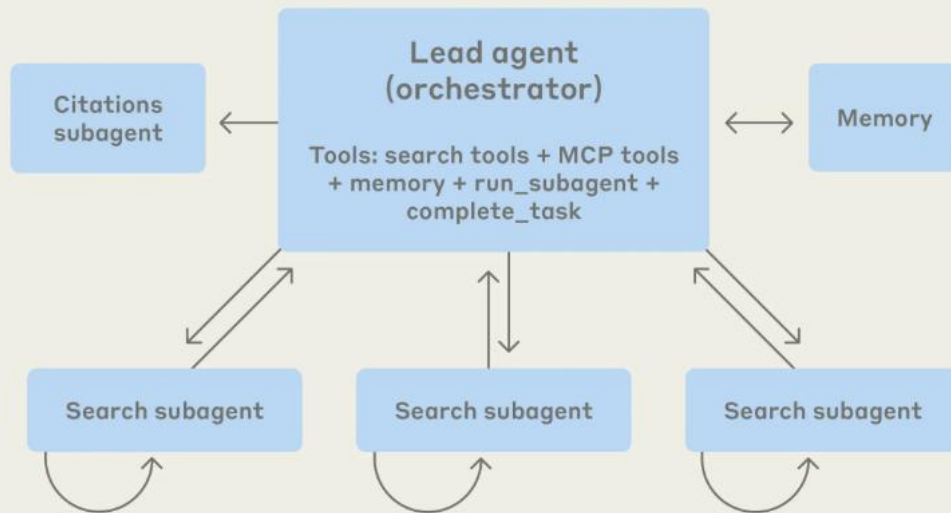


what are all the companies in the united states working on AI agents in 2025? make a list of at least 100. for each company, include the name, website, product, description of what they do, type of agents they build, and their vertical/industry.

User request

Final report

Multi-agent research system



작동 방식: 질의에서 최종 보고서까지

- **1) 질의 분해 (Decomposition)**

- 리드 에이전트가 사용자의 질문을 분석해 연구 계획 수립
- 예를 들어, "S&P 500 정보 기술 부문 기업들의 모든 이사회 구성원을 식별하라"는 질의가 주어지면, 리드 에이전트는 이를 '1) S&P 500 정보 기술 부문 기업 목록 식별', '2) 각 기업의 이사회 구성원 정보 검색'과 같은 여러 하위 작업으로 분해

- **2) 병렬 조사 (Parallel Investigation)**

- 여러 하위 에이전트가 동시에 웹 검색 등 도구를 사용해 정보 수집
- 각 하위 에이전트는 할당된 작업을 수행하기 위해 검색 도구를 반복적으로 사용
- 웹 검색을 수행하고, 도구 사용 결과를 평가하며, 이 과정을 통해 얻은 핵심적인 발견 사항을 정리

- **3) 합성 (Synthesis)**

- 리드 에이전트가 결과를 종합, 분석 후 필요시 추가 조사 지시
- 정보가 부족하거나 새로운 질문이 발생하면, 리드 에이전트는 새로운 하위 에이전트를 생성하여 추가 조사를 지시하거나 기존 전략을 수정하여 연구를 심화
- 이 반복적인 과정은 연구가 만족스러운 수준에 도달할 때까지 지속

- **4) 출처 표기 (Attribution)**

- 연구 루프가 성공적으로 종료되면, 리드 에이전트가 수집하고 종합한 모든 결과물은 전용 **인용 에이전트(CitationAgent)**에게 전달
 - ✓ 해당 에이전트는 최종 보고서와 원본 소스 문서들을 면밀히 검토하여, 보고서 내의 모든 주장이 정확한 출처에 근거하도록 구체적인 인용 위치를 식별하고 삽입

결과: 압도적 성능, 그러나 높은 비용

- **+90.2% 성능 향상**
 - 단일 에이전트 대비 내부 연구 평가에서 더 높은 점수 기록
- **x15 토큰 사용량**
 - 일반 채팅 대비 15배 많은 토큰 소비. 높은 성능은 막대한 계산 자원 투입의 결과
- **80% 성능 기여**
 - 성능 차이의 80%는 오직 '토큰 사용량'만으로 설명 가능. 이는 '더 똑똑한' AI라기보다 막대한 토큰 예산을 효과적으로 사용하여 문제를 해결하는 '경제적' 해결책 의미
 - ✓ 범용 인공지능(AGI)으로 가는 길이 더 효율적인 알고리즘(더 나은 추론)을 통하는 것인가, 아니면 대규모 계산을 적용하기 위한 더 나은 아키텍처(더 나은 엔지니어링)를 통하는 것인가? Anthropic의 시스템은 후자의 강력한 사례를 제시

한계: 다중 에이전트의 그림자

- **경제성**
 - 높은 비용으로 인해 고부가가치 작업에만 적용 가능
- **복잡성**
 - 에이전트 간 조정 오버헤드가 크고, 사소한 오류가 시스템 전체에 치명적일 수 있음
- **병목 현상**
 - 현재의 '동기식' 실행 방식은 병렬 처리의 장점을 완전히 살리지 못함
- **부적합한 과제**
 - 모든 에이전트가 동일한 컨텍스트를 공유해야 하는 '코딩' 같은 작업에는 부적합

차원	다중 에이전트 시스템	단일 에이전트 시스템
적합 과제	병렬화 가능한 '너비 우선' 탐색 (예: 시장 조사, 문헌 검토)	순차적인 '깊이 우선' 탐색 (예: 코딩, 단계별 증명)
핵심 장점	속도, 철저함, 결함 격리	컨텍스트 일관성, 단순함
핵심 단점	비용, 복잡성, 조정 오버헤드	경로 의존성, 단일 장애점

- '단일 vs 다중'은 잘못된 이분법이다.
- 미래는 과제의 성격에 따라 최적의 아키텍처를 동적으로 선택하는 하이브리드 시스템에 있다.
- 결국, AI 에이전트 시스템의 발전은 더 나은 '추론'뿐만 아니라, 더 나은 '엔지니어링'을 통해 이루어진다.

LangChain의 중재

- 두 개의 상반된 시각
 - Cognition AI: "멀티 에이전트는 위험하다. 만들지 말라." (코딩 실패 사례)
 - Anthropic: "멀티 에이전트는 강력하다. 우리는 성공했다." (리서치 성공 사례)
- LangChain의 관점: "누가 옳은가"가 아닌 "언제, 어떻게"의 문제
 - 두 주장은 모순이 아니라, 문제의 다른 측면을 보고 있을 뿐이다.
 - 성공과 실패를 가르는 가장 중요한 열쇠는 바로 **컨텍스트 엔지니어링**에 있다.

컨텍스트 엔지니어링 (Context Engineering)

- 정의
 - 에이전트가 매 순간 '정확히 필요한 만큼의 올바른 컨텍스트'를 갖도록 시스템을 설계하는 기술.
- 컨텍스트 엔지니어링으로 본 두 회사의 사례
 - Cognition의 실패: 각 에이전트에게 적절한 컨텍스트를 제공하는 데 실패
 - Anthropic의 성공: 리드 에이전트가 컨텍스트를 중앙에서 통제하고 효과적으로 분배하는 데 성공
- 결론
 - 에이전트 시스템의 성능은 컨텍스트 엔지니어링 수준에 달려있다

실용적인 판단 기준: "읽기(Read)" vs "쓰기(Write)"

작업 유형	"읽기(Read)" 중심 작업	"쓰기(Write)" 중심 작업
정의	여러 소스에서 정보를 수집, 분석, 요약	공유된 하나의 결과물을 동시에 수정, 변경
예시	논문 리서치, 웹 검색, 시장 분석	공동 코딩, 데이터베이스 수정
멀티 에이전트	적합 (Good)	부적합 (Bad)
이유	• 병렬 처리로 속도 향상 • 일부 실패가 전체에 영향 X	• 데이터 충돌 위험 매우 높음 • 일관성 유지 비용이 막대함
관련사례	Anthropic의 리서치 기능	Cognition의 코딩 실패 사례

현명한 개발자를 위한 조언

- 문제부터 분석하라: “읽기”인가, “쓰기”인가?
 - 작업의 성격이 멀티 에이전트 아키텍처 선택의 첫걸음이다.
 - 코딩과 연구는 모두 읽기와 쓰기를 포함하지만, 강조하는 측면은 서로 다름
 - ✓ 읽기 동작이 쓰기 동작보다 본질적으로 병렬화가 더 용이
 - ✓ “쓰기”에서의 충돌이 “읽기”에서의 충돌보다 더 나쁜 결과를 초래
- 컨텍스트 엔지니어링에 집중하라.
 - 성공적인 시스템은 결국 컨텍스트를 얼마나 잘 관리하느냐에 달려있다.
- 만병통치약은 없다.
 - 멀티 에이전트는 '읽기' 중심의 병렬 작업에서 강력한 도구이지만, 모든 문제의 해결책은 아니다. 문제에 맞는 최적의 아키텍처를 선택하는 것이 중요하다.

- 다중 에이전트 시스템의 장점 🚀
 - 여러 독립적인 방향을 **동시에 탐색**하는 연구나 분석 작업
 - 한 명의 에이전트가 가진 '작업 기억 용량(컨텍스트 창)'의 한계를 **팀 전체의 용량으로 극복**
 - 단일 에이전트로는 처리하기 어려운 복잡하고 방대한 너비 우선 질의에 특히 탁월
- 경제적 고려사항 💰
 - 다중 에이전트는 더 높은 성능을 위해 더 많은 토큰 사용량을 활용하는 것
 - 다중 에이전트 시스템은 여러 에이전트를 동시에 실행하므로 토큰 사용량이 많아져 비용이 증가
 - 성과 향상으로 얻는 가치가 증가하는 비용을 충분히 상쇄할 만큼 중요한 작업에 사용하는 것이 경제적으로 타당
- 부적합한 작업과 한계 ⚠️
 - **높은 상호 의존성**: 모든 에이전트가 동일한 정보를 실시간으로 공유하는 경우
 - **병렬화가 어려운 작업**: 코딩 작업 같이 독립 시행이 어려운 경우
 - **실시간 협업 능력 부족**: 현재 LLM 에이전트들은 다른 에이전트와 실시간으로 작업을 조율하고 위임하는 능력이 아직 미숙

Thank you!